

Záródolgozat

Metroidvania játék

Készítette:

Nagy-Huszár Virág

Huszár Imre László

Tartalomjegyzék

Bevezetés, a témánk bemutatása és céljaink	3
Mi az a Metroidvania játék?	3
Mi a célunk a projekttel?	3
Fejlesztői dokumentáció	4
Kialakított adatszerkezetek	4
Telepítés és indítás lépései	8
A program tipikus, egyedi, különleges vagy érdekesebb algoritmusai	10
A játék felépítése, logikai háttér, története és érdekességek	64
Tesztelés leírása, folyamata és tesztdokumentáció	69
Továbbfejlesztési lehetőségek	84
Felhasználói dokumentáció	87
A program célja és lényegesebb funkciói	87
Szükséges hardverek és szoftverek	87
Hibaelhárítás és gyakori kérdések	88
Telepítés és indítás lépései	89
A program részletes bemutatása	90
A játék és annak menete	92
Csapatmunka megvalósítása	93
Összefoglalás	97
Ábrajegyzék	98
Irodalomjegyzék	99

Bevezetés, a témánk bemutatása és céljaink

A záródolgozat megírása közben többször felmerült bennem, hogy miért kezdtük egyáltalán ezt az őrült ötletet el? Aztán rájöttem, hogy senki se erőlteti ránk, hogy mekkora játékot csináljunk vagy éppen, hogy pontosan mit csináljunk. A Metroidvania játék, mint ötlet, Team Cherry 2025. szeptemberében kijött (Hollow Knight: Silksong) játékuknak köszönhető. Alapvetően egy aranyos macskát terveztünk, de a tervező (Virág) beleturmixolta a Transformers sorozatok, képregényeket is. Így kötöttünk ki a transcica (jelenlegi adatbázisunk neve, köszi Imi) Metroidvaniánál. Egy hajnali 4 órai ötletből fogunk csinálni egy felvállalható kódot.

Mi az a Metroidvania játék?

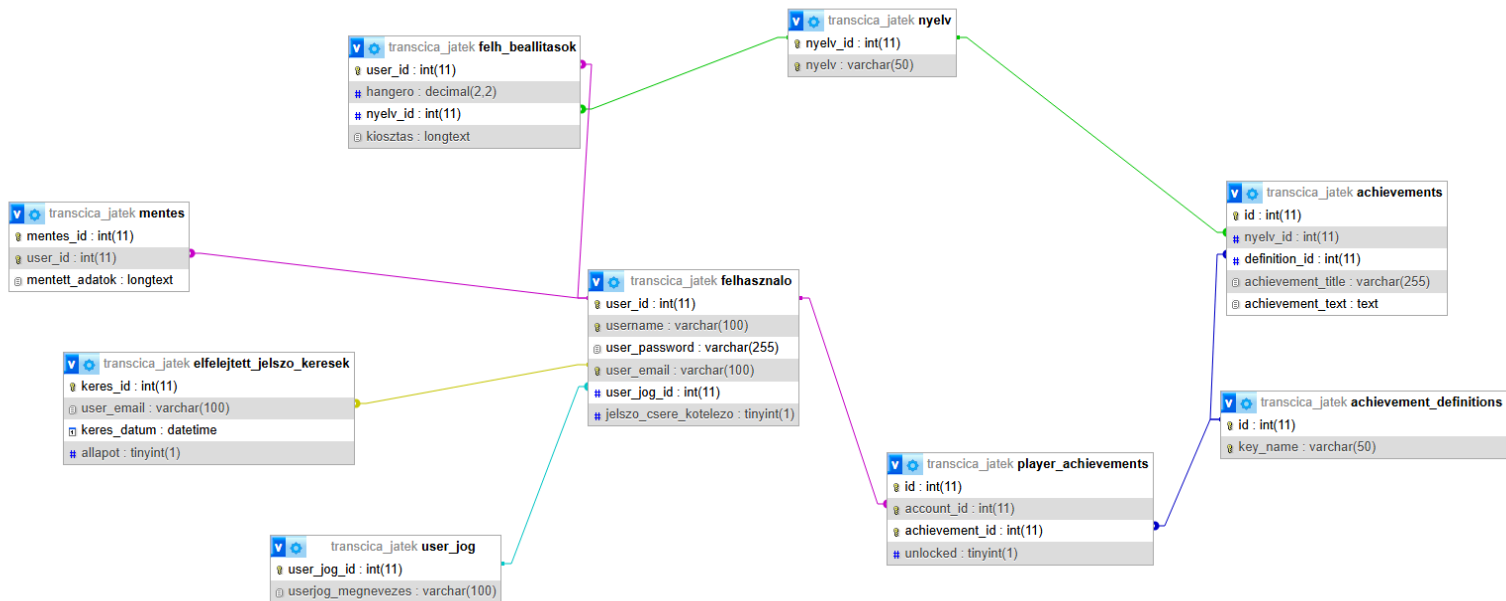
“A Metroidvania az akció-kalandjátékok egy alműfaja, amely a nemlineáris felfedezésre és az irányított fejlődésre összpontosít, ahol bizonyos területekre való belépéshez kulcsfontosságú tárgyakat kell megszerezni. A kifejezés a Metroid és a Castlevania videojáték-sorozatok nevének összevonásával jött létre, a Metroid (1986), a Castlevania II: Simon's Quest (1987), a Super Metroid (1994) és a Castlevania: Symphony of the Night (1997) mintáin alapulva.” - Forrás: Wikipédia
Több részletekért ajánlott kipróbálni egy Metroidvania játékot, megtekinteni az interneten egy végigjátszást vagy jobban utáni olvasni.

Mi a célunk a projekttel?

A komfortzónánkból ki szeretnénk menni a projekttel és kipróbálni, hogy milyen egy játékot végig fejleszteni egy rövidebb időszak alatt. Külső library-k használatát, játék tervezést és development pipeline-ok használatának megtanulását. Előre láthatóan vizualitás és hangtechnika előkészítését és tervezését is nekünk kell megcsinálni. A munka világában is számíthatunk arra, hogy a munkakörünkön kívül kell tevékenykednünk, ezért nagyobb körök kitesztelése, limitjeink megismerése és kezelése is kritikus.

Fejlesztői dokumentáció

Kialakított adatszerkezetek



1.ábra: SQL adatszerkezet

A program többféle adatszerkezetet használ, mert a játék nem csak egy egyszerű weboldalból áll, hanem tartalmaz felhasználókezelést, mentési rendszert, beállításokat, többnyelvű szöveg kezelést és Kaboom.js alapú játékmenetet is.

Az egyik legfontosabb adatszerkezet az adatbázisban található felhasználói adatok rendszere. A **felhasznalo** tábla tárolja a játékosok alapadatait, például a felhasználónevet, e-mail címet, jelszót és jogosultsági szintet. A jelszavakat nem egyszerű szöveggént tároljuk, hanem bcrypt segítségével hashelve, így biztonságosabban kezelhetők. A jogosultságokat külön **user_jog** tábla alapján kezeljük, így a program meg tudja különböztetni a vendég, normál felhasználó és adminisztrátor szerepköröket.

A beállítások tárolása szintén külön adatszerkezetként jelenik meg. A játékoshoz tartozó hangerő, nyelv, irányítási gombok és mobil mód menthetőek. Vendég

felhasználó esetén ezek az adatok főként `localStorage`-ban vannak tárolva, míg bejelentkezett felhasználónál az adatbázisból is visszatölthetők. Ez azért fontos, mert így a felhasználó saját beállításai később is megmaradnak.

A mentési rendszer összetettebb JSON alapú adatszerkezeteket használ. A mentésekben tároljuk például a játékos aktuális szobáját, mentési pontját, életerejét, megszerzett képességeit, kulcsait, valamint a világ állapotát. A `world_interactions` objektum például azt tárolja, hogy egy adott pályaelem vagy esemény megtörtént-e már. Ilyen lehet egy törhető fal, beszakadó talaj, világítótorony bekapcsolása, bónusz élet felvétele vagy a Crystal City kapujának kinyitása. Ennek köszönhetően a játék világa nem áll vissza alapállapotba minden szoba váltáskor.

Példa a `world_interactions` jellegű adatszerkezetre:

```
'world_interactions', JSON_OBJECT(  
    'mitteous-plateau_breakable-ground1', false,  
    'Iacon_breakable-ground1', false,  
    'Iacon_breakable-ground2', false,  
    'Iacon_breakable-wall1', false,  
    'Iacon_breakable-wall2', false,  
    'Iacon_breakable-wall3', false,  
    'Smelting-pits_breakable-wall1', false,  
    'lighthouse-on', false,  
    'lighthouse-sea-of-flowers-cutsscenes', false,  
    'crystal-heart-open_lock', false,  
    'crystal-city-key', false,  
    'crystal-heart-lava-protection', false,  
    'bonus-hp-1', false,  
    'bonus-hp-2', false,  
    'bonus-hp-3', false  
) ,
```

A nyelv kezeléshez JSON fájlokat használunk. A magyar és angol szövegek külön mappában vannak tárolva, például `hungarian` és `english` néven. A frontend nem fixen beégetett szövegeket használ, hanem API-n keresztül lekéri a megfelelő nyelvi fájlt. Ez rugalmasabbá teszi a rendszert, mert új nyelv hozzáadásához nem kell átírni az egész programot, elég új JSON fájlokat létrehozni.

```

  languages
  english
    {} achievements.json
    {} admin_panel.json
    {} beallitas_menu.json
    {} bejelentkezes_regisztracio.json
    {} credits.json
    {} exit.json
    {} in_game_menu.json
    {} kaboomBetolto.json
    {} main_menu.json
    {} mentes.json
    {} start_game.json
    {} suti_modal.json
    {} szoba_nevek.json
    {} tutorial.json
  hungarian
    {} achievements.json
    {} admin_panel.json
    {} beallitas_menu.json
    {} bejelentkezes_regisztracio.json
    {} credits.json
    {} exit.json
    {} in_game_menu.json
    {} kaboomBetolto.json
    {} main_menu.json
    {} mentes.json
    {} start_game.json
    {} suti_modal.json
    {} szoba_nevek.json
    {} tutorial.json

```

2.ábra: Nyelvkezelés JSON-ök

A mappa struktúránk

Vizsgaremek-Metroidvania-game

- backend
 - __ tests __
 - api
 - coverage => ez a jesté, ennek a részleteibe nem megyek be
 - game_json
 - maps
 - http
 - img
 - starting_screen
 - languages
 - english
 - hungarian
 - sql
- frontend
 - audio
 - bootstrap => ez a bootstrapé, ennek a részleteibe nem megyek be
 - css
 - cutscenes
 - html
 - images
 - UI
 - achivement_icons
 - maps
 - other
 - sprites
 - javascript
 - jatek
 - entitások
 - szobak
 - libraries

Telepítés és indítás lépései

A fentiek alapján a gépünk le tudja futtatni a programunkat, akkor a következőképpen kell telepíteni és elindítani a játékot:

1. Telepítsük a Visual Studio Code-ot vagy más hasonló program futtatásra alkalmas applikációt.
2. Telepítsünk egy lokális adatbázis rendszert kezelő applikációt (pl.: XAMPP, Oracle developer, stb.).
3. Töltsük le a Node.js-t a <https://nodejs.org/en> oldalról.

(A gépeken feltételezhetően ez az első három lépés már meg van csinálva, de ellenőrizni kell)

4. Töltsük le a Githubról a záradolgozatot a következő linkről:
<https://github.com/nagyhuszarvirag/Vizsgaremek-Metroidvania-game.git>
5. A tömörített mappát bontsuk ki és helyezzük egy számunkra könnyen megtalálható mappába.
6. Az első lépésben letöltött (,mostantól Visual Studio Code) applikációban megnyitjuk a kibontott mappát.
7. A Visual Studio Code-ban nyissunk egy új terminált.
8. A terminálba írjuk bele, hogy `npm init` abban az esetben, ha nincs `package.json` fájl
9. A terminálba írjuk bele, hogy `cd backend`
 - a. Tesztelés esetén a következőket is bele kell írni a terminálba:
 - i. `npm install --save-dev jest`
 - ii. `npm install --save-dev jest-axios`
 - iii. `npm i jest-environment-jsdom`
 - iv. `npm i babel-jest`
 - v. `npm install --save-dev @babel/core @babel/preset-env babel-jest`
 - vi. Tesztek futtatása: `npm run test`
 - b. Játék esetén a következőket kell még a terminálba beírni (, ha először futtatni a programot és nincsen minden telepítve):
 - i. `npm install`
 - ii. `npm install bcrypt`
 - c. Az `npm run start`-ot beírjuk a terminálba a program futtatásáért

10. Miután ez megtörtént a XAMPP vagy az általunk választott adatbázist elindítjuk lokálisan és a záródolgozat backend/sql mappájában található adatbázis SQL-t betöltjük.
11. A játékot ezek után lehet futtatni a következőkkel (ezt is a Visual Studio Code terminálba kell beleírni): `npm run start`
12. A következő linken lesz található az oldal ettől a ponttól: <http://127.0.0.1:3000>
13. Az oldal használható vendégként és fiókkal is, viszont vendégként kevesebb dolog elérhető, és teljes kilépés után minden mentés elvész.
14. Ha adminként szeretnénk a programot használni, akkor a következőkkel be lehet jelentkezni/regisztrálni:
 - a. Username: Admin Password: Admin_123
 - b. nagyhuszarvirag@gmail.com email címmel fiókot csinálunk
 - c. imre.huszar6@gmail.com email címmel fiókot csinálunk
15. Használati lépésekért a következő fő pontot olvassa el: "A program részletes bemutatása"

A program tipikus, egyedi, különleges vagy érdekesebb algoritmusai

```
export function oldalTakarito() {
  const Zenemegtarto = document.getElementById("zenemarad"); //Mindig
  akarunk zenét
  if (Zenemegtarto) {
    Array.from(document.body.children).forEach((child) => {
      //Megnézi az összes gyereket és eltávolítja őket, kivéve a zenét
      if (child !== Zenemegtarto) {
        child.remove();
      }
    });
  } else {
    //Teljes törlés, ha nincs zene
    document.body.innerHTML = "";
  }
}
```

Az oldalTakarito függvény teszi lehetővé számunkra, hogy a menük között könnyebben tudjunk haladni és több html oldal helyett egy html oldallal kellett így dolgoznunk. Alapvetően mindent törölünk, ha nincs zene, de ha van, akkor a zenén kívül minden mást törölünk. A zenét azért nem töröljük és töljük újra minden egyes menünél, mert mindig megszakadni a zene és mindig újra kellene engedélyezni a zene lejátszását. Ez az élményt és immerziót elrontaná, amit elkerültünk egy egyszerű if-else ággal.

```
export function visszaGomb(vissza) {
  let Gomblehetsosegek = document.createElement("button");

  Gomblehetsosegek.innerText = vissza;
  Gomblehetsosegek.classList.add("gombok");
  Gomblehetsosegek.id = "vissza_gomb";

  Gomblehetsosegek.addEventListener("click", async () => {
    await createMainMenu();
  });

  return Gomblehetsosegek;
}
```

A másik függvény, ami kritikus az menüpontok közötti mozgáshoz. A vissza gombra nyomva minden esetben a főmenünél kötünk ki. A függvény változóban várja a “*vissza*” szövegét, ami a nyelvkezelés miatt szükséges (Vissza/Back). Visszatérési értéke pedig a kész gomb, amit appendChild függvénnyel hozzáadjuk a menüpont jó részéhez.

```
export function dekor_vonal_blokkal() {
  const sor = document.createElement("div");
  sor.classList.add("row", "dekor_vonal");

  for (let i = 0; i < 3; i++) {
    let span = document.createElement("span");
    span.classList.add("kocka");
    span.innerText = "■";
    sor.appendChild(span);
  }

  return sor;
}
```

A dekor vonal függvény bár nem része a “kritikus az oldalhoz való funkcionálásához”, de esztétikailag sokat dob az oldalon és segíti az átláthatóságot is. A függvény létrehoz egy sor (div), amibe belerakjuk a három darab ■ blokkot és a blokkok a “*kocka*” classt kapják és a div a “*dekor_vonal*” classt kapja. És ezek után visszaadjuk a sor, hogy appendChild függvénnyel könnyen a jó helyre belerakhassuk. A csoda, viszont nem itt történik, hanem a CSS-ben:

```

.dekor_vonal {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 100%;
  margin: 40px 0;
  position: relative;
}

.dekor_vonal::before,
.dekor_vonal::after {
  content: "";
  flex: 1;
  height: 2px;
  background-color: #ffffff;
}

.kocka {
  position: relative;
  top: -20px;
  left: 8px;
  width: 10px;
  height: 10px;
  transform: rotate(45deg);
}

```

A `dekor_vonal` class létrehozza a fehér csíkot, ami a blokkok körül található és a `kocka` class pedig a blokkokat elfordítja 45°-kal és pontosan egy vonalba teszi a vonallal (ezt nagyrészt manuális teszteléssel tudtuk megoldani, mert nem találtunk olyan függvényt ami pontosan ezt csinálja).

```

const nyelvMappaUtvonala = path.join(__dirname, "../languages/");

//? GET /api/nyelv_alapjan_JSON_olvasas
router.get(
  "/nyelv_alapjan_JSON_olvasas/:nyelv/:fajl",
  async (request, response) => {
    try {
      const { nyelv, fajl } = request.params;

      // Biztonsági ellenőrzés: csak .json kiterjesztésű fájlok
      engedélyezése (Mert más fájlokat nem akarunk olvasni és támadások
      elkerülésének érdekében van itt)
      if (!fajl.endsWith(".json")) {
        return response.status(400).json({
          success: false,
          message: "Csak JSON fájlokat fogadunk el!",
        });
      }

      let nyelvutvonal = "hungarian";

      switch (nyelv) {
        case "1":
          nyelvutvonal = "hungarian";
          break;
        case "2":
          nyelvutvonal = "english";
          break;
        default:
          return response.status(400).json({
            success: false,
            message: "Érvénytelen nyelv!",
          });
      }

      const fajlUtvonal = path.join(
        nyelvMappaUtvonala,
        nyelvutvonal,
        "/",
        fajl,
      );

```

```

        const fajlTartalom = await fsPromises.readFile(fajlUtvonal,
"utf8");
        const JSONAdatok = JSON.parse(fajlTartalom);

        response.status(200).json({
            success: true,
            data: JSONAdatok,
        });
    } catch (error) {
        console.error("GET error:", error);

        response.status(404).json({
            success: false,
            message: "A JSON fájl nem létezik!",
        });
    }
},
);

```

Ez az API az egyik (vagy a) legkritikusabb és feltételezhetően leggyakrabban használt függvényünk. Az API várja paraméterként a user beállított nyelvét és a fájl nevét. Elsősorban ellenőrizzük, hogy .json végű fájlt kérünk le, hogy később a visszaadásnál ne legyen probléma. Ezek után beállítjuk a nyelvnek az útvonalát (ami alapvetően magyar) és ha ismeretlen nyelv id-t adunk meg, akkor visszaküldjük ezt, mint hibát. Ezek után a fájlnek az útvonalát összerakjuk példa:

```

const fajlUtvonal = path.join(
    nyelvMappaUtvonala,
    nyelvutvonal,
    "/",
    fajl,
);

```

Adatokkal:

```

const fajlUtvonal = path.join(
    "../languages/",
    "hungarian",
    "/",
    "main_menu.json",
);

```

Ezek után `fsPromises.readFile` függvénnyel kiolvassuk a fájl tartalmát és `JSON.parse` függvénnyel egy visszaadható JSON-né tesszük. Ezután a `response.json`-nek a `data` részén visszaadjuk a szoba szövegét és felhasználjuk, hogy a megfelelő szöveget a megfelelő nyelven és helyen írjuk ki.

```
export async function cutscene_kezeles(k, scene_name, nextScene = null,
callback = null) {
  const data = await fetchData(
    "http://127.0.0.1:3000/api/nyelv_alapjan_JSON_olvasas/" +
    settings.nyelv + "/kaboomBetolto.json"
  );

  const container = document.body;

  const zene = document.getElementById("zenemarad");

  let elozoMuted = true;
  let elozoVolume = 1;

  if (zene) {
    elozoMuted = zene.muted;
    elozoVolume = zene.volume;
    zene.pause();
  }

  const wrapper = document.createElement("div");
  wrapper.id = "cutscene_wrapper";
  wrapper.style.position = "fixed";
  wrapper.style.inset = "0";
  wrapper.style.width = "100vw";
  wrapper.style.height = "100vh";
  wrapper.style.zIndex = "99999";
  wrapper.style.backgroundColor = "black";
  wrapper.style.display = "flex";
  wrapper.style.justifyContent = "center";
  wrapper.style.alignItems = "center";

  const video = document.createElement("video");
  video.id = "videok";
  video.autoplay = true;
  video.playsInline = true;
  video.style.position = "absolute";
```

```

video.style.inset = "0";
video.style.width = "100vw";
video.style.height = "100vh";
video.style.objectFit = "cover";
video.style.zIndex = "1";
video.style.backgroundColor = "black";

const source = document.createElement("source");
source.src = "../../cutscenes/" + scene_name + ".mp4";
source.type = "video/mp4";

video.appendChild(source);

const skipButton = document.createElement("button");
skipButton.textContent = data.data.skip;
skipButton.style.position = "absolute";
skipButton.style.top = "20px";
skipButton.style.left = "20px";
skipButton.style.zIndex = "2";
skipButton.style.padding = "12px 20px";
skipButton.style.fontSize = "20px";
skipButton.style.cursor = "pointer";
skipButton.style.border = "2px solid black";
skipButton.style.background = "white";
skipButton.style.color = "black";

wrapper.appendChild(video);
wrapper.appendChild(skipButton);
container.appendChild(wrapper);

let closed = false;

function cleanupAndGo() {
  if (closed) return;
  closed = true;

  wrapper.remove();

  window.focus();

  const canvas = document.querySelector("canvas");
  if (canvas) {
    canvas.tabIndex = 0;
  }
}

```



```

        canvas.focus();
    }

    if (nextScene !== null) {
        k.go(nextScene);
    }

    if (callback) {
        callback();
    }

    if (zene) {
        zene.pause();
        //zene.src = "../audio/the_humbling_river.mp3";
        zene.currentTime = 0;
        zene.muted = elozoMuted;
        zene.volume = elozoVolume;

        if (!elozoMuted) {
            zene.play().catch(err => {
                console.log("Zene indítás hiba:", err);
            });
        }
    }
}

video.addEventListener("ended", cleanupAndGo);
skipButton.addEventListener("click", cleanupAndGo);

k.onKeyPress("enter", () => {
    cleanupAndGo();
});

video.load();
}

```

A cutscene kezelés a játék hangulatának és alap környezetének megadója. Alapvetően 2 paraméterrel dolgozik, “*k*” a kaboom játékkörnyezetet adja át a függvénynek, hogy a játék kontextuson belül érzékelje az onKeyPress-t (, ami egy

fancy addEventListener), a `“scene_name”`-t, ami a videó nevét határozza meg, mint például:

```
cutscene_kezeles(k, "rigor_morphis");
```

Így az útvonala a jelenetnek ebben az esetben a következő lesz:

```
source.src = "../../cutscenes/rigor_morphis.mp4";
```

Ezekén kívül még 2 darab változó található, ami egyből általánosítva lett, mert nem tudtuk, hogy a végén csak 1 eset miatt fog kelleni, vagy több is lesz. A `“nextScene”` változó eredetileg az intro miatt kellett csak, de a játék lezárására is azt használjuk, még a `“callback”` változót csak a temető szobában használjuk a sound effektek helyes kezelése miatt.

A `“data”` változóban meghívjuk a videó átugrására való gomb szövegét (Skip cutscene vagy Jelenet átugrása) és a `“skipButton”` gomb szöveggént feltöltjük. Ha erre a gombra vagy az ENTER-re nyomunk, akkor a `“cleanupAndGo”` belső függvény meghívásra kerül.

Ezek után a `“video”` változót létrehozunk, belerakjuk az src-t és más tulajdonságait és belerakjuk a body-ba és elkezdjük a videó lejátszást. Amikor a videónak vége van, akkor a `“cleanupAndGo”` belső függvény meghívásra kerül ugyanúgy.

Abban az esetben, ha van `“zene”` vagy más hang effektus, azokat erre az időszakra némítjuk.

A `cleanupAndGo` belső függvény. Minden esetben, a jelenet lezárására van meghívva. Elsősorban ellenőrizni, hogy már be van-e zárva a `closed` változóval. Ha már be van zárva, akkor a függvény lezárul. Viszont, abban az esetben, ha nincs lezárva a függvény, akkor törli a videót tároló `wrapper` változót és `nextScene` változó esetén meghívja a

```
k.go(nextScene);
```

segítségével elmegy a következő jelenetbe (intro vagy ending). A felsorolt eseteken kívül ez nem kerül meghívásra. Ezen kívül a `callback` változót ellenőrzi, hogy van-e, és ha van, akkor meghívja az. Ezt konkrétan a temetőben található hangeffektek kezelése miatt van csak.

```
import { sebzesAdas } from "./hp_kezelo.js";

export function ScrapletLetrehozas(k, x, y, player, patrolRange = 120)
{
    const scraplet = k.add([
        k.sprite("scraplet"),
        k.pos(x, y),
        k.anchor("center"),
        k.area({
            shape: new k.Rect(k.vec2(2, 6), 28, 24),
        }),
        k.body(),
        "enemy",
        "scraplet",
    ]);

    scraplet.hp = 4;
    scraplet.dead = false;
    scraplet.attacking = false;
    scraplet.chasing = false;
    scraplet.pihenés = false;
    scraplet.speed = 45;
    scraplet.damageCooldown = false;
    scraplet.hurtCooldown = false;
    scraplet.irany = 1;
    scraplet.turnCooldown = false;
```

```

scraplet.attackRange = 80;
scraplet.attackMode = false;
scraplet.animState = "idle";

scraplet.spawnX = x;
scraplet.spawnY = y;
scraplet.patrolRange = patrolRange;

function animValtas(animNev) {
    if (scraplet.animState === animNev) return;

    scraplet.animState = animNev;
    scraplet.play(animNev);
}

//scraplet.play("idle");

animValtas("idle");

function randomPihenes() {
    if (scraplet.dead || scraplet.chasing) return;

    scraplet.pihenes = true;
    //scraplet.play("idle");
    animValtas("idle")

    const pihenesIdo = 2 + Math.random();

    k.wait(pihenesIdo, () => {
        if (!scraplet.exists() || scraplet.dead ||
scraplet.chasing) return;

        scraplet.pihenes = false;

        if (Math.random() < 0.5) {
            scraplet.irany *= -1;
        }

        randomPihenes();
    });
}

k.wait(2, () => {

```

```

        if (scraplet.exists()) {
            randomPihenes();
        }
    });

    scraplet.onUpdate(() => {
        if (scraplet.dead) return;
        if (scraplet.attacking) return;
        if (scraplet.hurtCooldown) return;

        const tavolsagX = player.pos.x - scraplet.pos.x;
        const tavolsagY = Math.abs(player.pos.y - scraplet.pos.y);
        const absTavolsagX = Math.abs(tavolsagX);
        const tavSpawnTol = Math.abs(scraplet.pos.x - scraplet.spawnX);

        const chaseStartRange = 180;
        const chaseEndRange = 260;
        const chaseYRange = 70;

        if (!scraplet.chasing) {
            if (absTavolsagX <= chaseStartRange && tavolsagY <=
chaseYRange) {
                scraplet.chasing = true;
            }
        } else {
            if (absTavolsagX > chaseEndRange || tavolsagY > chaseYRange
+ 30) {
                scraplet.chasing = false;
                scraplet.attackMode = false;
            }
        }

        if (scraplet.chasing) {
            scraplet.pihenes = false;

            const attackStartRange = 80;
            const attackEndRange = 105;
            const minimumTavolsag = 8;

            if (
                !scraplet.attackMode &&
                absTavolsagX <= attackStartRange &&
                tavolsagY < 50

```

```

    ) {
        scraplet.attackMode = true;
    }

    if (
        scraplet.attackMode &&
        (absTavolsagX > attackEndRange || tavolsagY >= 60)
    ) {
        scraplet.attackMode = false;
    }

    if (absTavolsagX > 14) {
        scraplet.irany = tavolsagX > 0 ? 1 : -1;
        scraplet.flipX = scraplet.irany < 0;
    }

    // attack anim közben is menjen tovább a player felé
    if (absTavolsagX > minimumTavolsag) {
        scraplet.move(scraplet.irany * scraplet.speed, 0);
    }

    if (scraplet.attackMode) {
        animValtas("attack");
    } else {
        animValtas("walk");
    }
} else {
    if (!scraplet.pihenes) {
        scraplet.flipX = scraplet.irany < 0;

        if (scraplet.pos.x <= scraplet.spawnX -
scraplet.patrolRange) {
            scraplet.irany = 1;
            scraplet.flipX = false;
        }

        if (scraplet.pos.x >= scraplet.spawnX +
scraplet.patrolRange) {
            scraplet.irany = -1;
            scraplet.flipX = true;
        }
    }
}

```

```

        scraplet.move(scraplet.irany * scraplet.speed * 0.5,
0);

        if (scraplet.curAnim() !== "walk") {
            scraplet.play("walk");
        }
    } else {
        if (scraplet.curAnim() !== "idle") {
            //scraplet.play("idle");
            animValtas("idle");
        }
    }
}

});

scraplet.onCollideUpdate("player", () => {
    if (scraplet.dead) return;
    if (scraplet.damageCooldown) return;
    if (scraplet.hurtCooldown) return;

    scraplet.damageCooldown = true;
    scraplet.attacking = true;

    //scraplet.play("attack");
    animValtas("attack");

    sebzsesAdas(k, player, 1);

    k.wait(0.35, () => {
        if (!scraplet.exists() || scraplet.dead) return;

        scraplet.attacking = false;

        if (scraplet.chasing || !scraplet.pihenes) {
            //scraplet.play("walk");
            animValtas("walk");
        } else {
            //scraplet.play("idle");
            animValtas("idle");
        }
    });

    k.wait(1.0, () => {

```

```

        if (!scraplet.exists()) return;
        scraplet.damageCooldown = false;
    });
});

k.onCollide("player_attack_hitbox", "scraplet", (hitbox, enemy) =>
{
    if (enemy !== scraplet) return;
    if (scraplet.dead) return;
    if (scraplet.hurtCooldown) return;

    scraplet.hurtCooldown = true;
    scraplet.hp -= 1;

    console.log("Scraplet HP:", scraplet.hp);

    if (scraplet.hp > 0) {
        //scraplet.play("hurt");
        animValtas("hurt")
        scraplet.move(scraplet.flipX ? 120 : -120, 0);

        k.wait(0.2, () => {
            if (!scraplet.exists() || scraplet.dead) return;

            scraplet.hurtCooldown = false;

            if (scraplet.chasing || !scraplet.pihenes) {
                //scraplet.play("walk");
                animValtas("walk");
            } else {
                //scraplet.play("idle");
                animValtas("idle");
            }
        });
    }

    if (scraplet.hp <= 0) {
        scraplet.dead = true;
        //scraplet.play("die");
        animValtas("die");

        k.wait(0.4, () => {
            if (scraplet.exists()) scraplet.destroy();
        });
    }
});

```



```

    });
  }
});

return scraplet;
}

```

A scrapletek a játékunknak az egyetlen egyszerű ellensége (rajtuk kívül csak 2 boss van) és az ellenség logika kezelésének alapjait rajtuk keresztül mutatom be.

A scraplet létrehozása egy külön függvényben történik, ahol megadjuk az alap tulajdonságait, mint például a pozícióját, életerejét, sebességét és az aktuális állapotát. A Kaboom.js komponens alapú rendszere miatt egy ellenség több különböző részből áll össze (sprite, pozíció, ütközés, fizika stb.).

```

const scraplet = k.add([
  k.sprite("scraplet"),
  k.pos(x, y),
  k.anchor("center"),
  k.area({
    shape: new k.Rect(k.vec2(2, 6), 28, 24),
  }),
  k.body(),
  "enemy",
  "scraplet",
]);

```

Ezután különböző változókkal írjuk le az ellenség állapotát:

```

scraplet.hp = 4;
scraplet.dead = false;
scraplet.attacking = false;
scraplet.chasing = false;
scraplet.pihenés = false;
scraplet.speed = 45;
scraplet.damageCooldown = false;
scraplet.hurtCooldown = false;
scraplet.irany = 1;
scraplet.turnCooldown = false;
scraplet.attackRange = 80;

```

```

scraplet.attackMode = false;
scraplet.animState = "idle";

scraplet.spawnX = x;
scraplet.spawnY = y;
scraplet.patrolRange = patrolRange;

```

A scraplet működésének alapja egy egyszerű állapotgép (state machine). Három fő állapota van:

- **járőrözés (patrol)** – ha a játékos nincs a közelben
- **üldözés (chase)** – ha a játékos elég közel kerül
- **támadás (attack)** – ha elérhető közelségbe kerül a játékos

Az `onUpdate` függvényben folyamatosan figyeljük a játékos távolságát, és ez alapján váltogatjuk az állapotokat:

```

const tavolsagX = player.pos.x - scraplet.pos.x;
const tavolsagY = Math.abs(player.pos.y - scraplet.pos.y);
if (!scraplet.chasing) {
    if (absTavolsagX <= chaseStartRange && tavolsagY <=
chaseYRange) {
        scraplet.chasing = true;
    }
}
const attackStartRange = 80;
const attackEndRange = 105;
const minimumTavolsag = 8;

    if (
        !scraplet.attackMode &&
        absTavolsagX <= attackStartRange &&
        tavolsagY < 50
    ) {
        scraplet.attackMode = true;
    }

```

Üldözés közben az ellenség a játékos irányába mozog:

```
scraplet.irany = tavolsagX > 0 ? 1 : -1;
scraplet.flipX = scraplet.irany < 0;

scraplet.move(scraplet.irany * scraplet.speed * 0.5, 0);
```

Ha elég közel kerül, támadó módba vált és animációt is váltunk:

```
if (
    !scraplet.attackMode &&
    absTavolsagX <= attackStartRange &&
    tavolsagY < 50
) {
    scraplet.attackMode = true;
}

if (
    scraplet.attackMode &&
    (absTavolsagX > attackEndRange || tavolsagY >= 60)
) {
    scraplet.attackMode = false;
}
```

A sebzés kezelése ütközés alapján történik. Amikor a scraplet hozzáér a játékoshoz, sebzést adunk neki, de cooldown-t használunk, hogy ne sebződjön folyamatosan:

```
scraplet.onCollideUpdate("player", () => {
    if (scraplet.dead) return;
    if (scraplet.damageCooldown) return;
    if (scraplet.hurtCooldown) return;

    scraplet.damageCooldown = true;
    scraplet.attacking = true;

    //scraplet.play("attack");
    animValtas("attack");

    sebzesAdas(k, player, 1);

    k.wait(0.35, () => {
```

```

        if (!scraplet.exists() || scraplet.dead) return;

        scraplet.attacking = false;

        if (scraplet.chasing || !scraplet.pihenes) {
            //scraplet.play("walk");
            animValtas("walk");
        } else {
            //scraplet.play("idle");
            animValtas("idle");
        }
    });

    k.wait(1.0, () => {
        if (!scraplet.exists()) return;
        scraplet.damageCooldown = false;
    });
});

```

```

}

```

A játékos támadását is kezeljük. Ha a scraplet találatot kap, csökkentjük az életerejét, és ha elfogy, meghal:

```

scraplet.hurtCooldown = true;
scraplet.hp -= 1;

console.log("Scraplet HP:", scraplet.hp);

if (scraplet.hp > 0) {
    //scraplet.play("hurt");
    animValtas("hurt")
    scraplet.move(scraplet.flipX ? 120 : -120, 0);

    k.wait(0.2, () => {
        if (!scraplet.exists() || scraplet.dead) return;

        scraplet.hurtCooldown = false;

```

Tarn

```

        if (scraplet.chasing || !scraplet.pihenes) {
            //scraplet.play("walk");
            animValtas("walk");

```

```

        } else {
            //scraplet.play("idle");
            animValtas("idle");
        }
    });
}

if (scraplet.hp <= 0) {
    scraplet.dead = true;
    //scraplet.play("die");
    animValtas("die");

    if (scraplet.tutorialZone &&
scraplet.tutorialZone.exists()) {
        scraplet.tutorialZone.destroy();
    }

    k.wait(0.4, () => {
        if (scraplet.exists()) scraplet.destroy();
    });
}

```

Összességében a scraplet egy egyszerű, de jól bővíthető ellenség típus, amely bemutatja az alap AI működést: érzékelés (távolság), döntés (állapotváltás) és cselekvés (mozgás, támadás, animáció). Ez a logika később a bossoknál egy összetettebb formában kerül felhasználásra.

A játékban két boss található: **Sparkeater** és **Tarn**. Mindkettő külön fájlban és külön létrehozó függvényben van kezelve, mert összetettebb logikával rendelkeznek, mint az egyszerű scrapletek. A bossoknak saját életerejük, támadásaik, cooldownjaik, arénájuk és jutalmuk van.

A Sparkeater létrehozásakor megadjuk az alap Kaboom komponenseket, majd külön tulajdonságokkal szabályozzuk az állapotát:

```
boss.hp = 30;
boss.speed = 75;
boss.dead = false;
boss.attacking = false;
boss.damageBoosted = false;
boss.attackCooldown = 0;
boss.grappleCooldown = 0;
boss.screamCooldown = 0;
boss.hitCooldown = false;
boss.fightActive = false;
boss.stuckTimer = 0;
boss.hopCooldown = 0;
```

A boss csak akkor aktív, ha a játékos az arénán belül van. Ezt a `playerArenabanVan()` függvény ellenőrzi:

```
function playerArenabanVan() {
    if (!arenaObj) return true;

    return (
        player.pos.x >= arenaObj.x &&
        player.pos.x <= arenaObj.x + arenaObj.width &&
        player.pos.y >= arenaObj.y &&
        player.pos.y <= arenaObj.y + arenaObj.height
    );
}
```

Sparkeater három fő támadással rendelkezik. Van egy közeli normál támadása, egy láncos/grapple támadása, illetve egy scream támadása. A scream nem közvetlenül sebez, hanem a következő támadást erősíti fel:

```
function damageErtek(alap) {
    if (!boss.damageBoosted) return alap;

    boss.damageBoosted = false;
    return alap * 2;
}
```

```

function grappleAttack() {
    if (boss.attacking || boss.dead) return;
    if (!boss.fightActive) return;

    boss.attacking = true;
    boss.grappleCooldown = 3.0;

    bossAnim("grapple");

    k.wait(0.25, () => {
        if (!boss.exists() || boss.dead || !boss.fightActive)
return;

        const irany = player.pos.x > boss.pos.x ? 1 : -1;
        boss.flipX = irany < 0;

        const startX = boss.pos.x + irany * 35;
        const startY = boss.pos.y - 8;

        const targetX = player.pos.x;
        const targetY = player.pos.y - 8;

        const dx = targetX - startX;
        const dy = targetY - startY;

        const tav = Math.sqrt(dx * dx + dy * dy);
        const angle = Math.atan2(dy, dx);

        const chainPieces = [];
        const pieceDistance = 18;
        const pieceCount = Math.max(1, Math.floor(tav /
pieceDistance));

        for (let i = 0; i < pieceCount; i++) {
            const px = startX + Math.cos(angle) * pieceDistance *
i;

            const py = startY + Math.sin(angle) * pieceDistance *
i;

            const chain = k.add([
                k.pos(px, py),
                k.sprite("sparkeater_chain"),

```

```

        k.anchor("center"),
        k.rotate(angle * 180 / Math.PI),
        k.opacity(1),
        k.z(20),
    ]);

    chain.play("active");
    chainPieces.push(chain);
}

const head = k.add([
    k.pos(targetX, targetY),
    k.sprite("sparkeater_chain_head"),
    k.anchor("center"),
    k.rotate(angle * 180 / Math.PI),
    k.opacity(1),
    k.z(21),
]);

head.play("active");

const grappleHitbox = k.add([
    k.pos(targetX - 22, targetY - 22),
    k.rect(44, 44),
    k.area(),
    k.opacity(0),
    "sparkeater_grapple_hitbox",
]);

k.wait(0.22, () => {
    chainPieces.forEach((piece) => {
        if (piece.exists()) piece.destroy();
    });

    if (head.exists()) head.destroy();
    if (grappleHitbox.exists()) grappleHitbox.destroy();
});

k.wait(0.8, () => {
    if (!boss.exists() || boss.dead) return;
    boss.attacking = false;
});

```



```
}
```

A támadások közötti választás az `onUpdate()` függvényben történik a cooldownok és a játékosról mért távolság alapján:

```
if (!boss.attacking) {
    boss.flipX = t.dx < 0;

    if (boss.screamCooldown <= 0 && boss.hp <= 18) {
        scream();
        return;
    }

    if (boss.grappleCooldown <= 0 && t.dist <= 150 && t.dist >
65 && t.absY <= 70) {
        grappleAttack();
        return;
    }

    if (boss.attackCooldown <= 0 && t.dist <= 65 && t.absY <=
55) {
        normalAttack();
        return;
    }

    if (t.dist > 45) {
        const irányX = t.dx > 0 ? 1 : -1;

        boss.move(irányX * boss.speed, 0);
        bossAnim("walk");

        return;
    } else {
        bossAnim("idle");
    }
}
```

Ha a játékos legyőzi Sparkeatert, akkor a mentésben eltároljuk, hogy a boss meghalt, a játékos megkapja a dash képességet és a Crystal City kulcsot:

```
mentett.bosses.Sparkeater = true;
mentett.ability_unlocked.dash = true;
mentett.world_interactions["crystal-city-key"] = true;
```

A második boss Tarn, aki nagyobb életerővel és más típusú támadásokkal rendelkezik:

```
boss.hp = 40;
  boss.speed = 65;
  boss.dead = false;
  boss.attacking = false;
  boss.fightActive = false;
  boss.resetting = false;

  boss.shootCooldown = 0;
  boss.stompCooldown = 0;
  boss.jumpCooldown = 0;
  boss.hitCooldown = false;
```

Tarn fő támadásai a lövés, a stomp támadás és az ugrásos mozgás. A lövésnél egy külön lövedék objektum jön létre, ami sebzi a játékost, ha eltalálja:

```
const bullet = k.add([
  k.pos(boss.pos.x + irany * 45, boss.pos.y - 10),
  k.sprite("tarn_bullet_sprite"),
  k.anchor("center"),
  k.area({
    shape: new k.Rect(k.vec2(-12, -8), 24, 16),
  }),
  k.move(k.vec2(irany, 0), 260),
  "tarn_bullet",
]);
```

A stomp támadás egy rövid ideig létező sebző zónát hoz létre:

```
const zone = k.add([
  k.pos(targetX, targetY + 20),
  k.sprite("tarn_stomp_sprite"),
  k.anchor("center"),
  k.scale(2),

  k.area({
    shape: new k.Rect(
      k.vec2(0, -5),
      40,
      40
    ),
  }),

  k.opacity(1),
  k.z(20),
  "tarn_stomp_zone",
]);
```

Tarnál külön érdekesség az `emergencyArenaReset()` függvény. Ha a boss rossz helyre kerül vagy lávával ütközik, akkor a program visszarakja az aréna biztonságos részére:

```
function emergencyArenaReset() {
    if (boss.resetting || boss.dead) return;

    boss.resetting = true;
    boss.attacking = false;

    //jobb oldalra visszarakás
    boss.pos.x = arenaObj.x + arenaObj.width - 100;
    boss.pos.y = arenaObj.y + arenaObj.height - 140;

    TarnAnimation(boss, "walk");

    if (boss.vel) {
        boss.vel.x = 0;
        boss.vel.y = 0;
    }

    //kis késleltetés után felugrás a felső rész felé
    k.wait(0.2, () => {
        if (!boss.exists() || boss.dead) return;

        boss.jump(650);
        boss.move(-260, 0);
    });

    k.wait(1.2, () => {
        if (!boss.exists() || boss.dead) return;
        boss.resetting = false;
    });
}
```

```
boss.onCollideUpdate("Lava_object", () => {
    emergencyArenaReset();
});
```

Tarn döntési logikája szintén az `onUpdate()` függvényben történik. A program figyeli a cooldownokat és a játékostól mért távolságot, majd ennek alapján választ támadást vagy mozgást:

```
if (boss.stompCooldown <= 0 && t.dist <= 280) {
    TarnAnimation(boss, "stomp");
    stompAttack();
    return;
}

if (boss.shootCooldown <= 0 && t.dist <= 320) {
    TarnAnimation(boss, "shoot");
    shootAttack();
    return;
}

if (boss.jumpCooldown <= 0 && t.dist > 180) {
    TarnAnimation(boss, "jump");
    jumpMove();
    return;
}

if (t.dist > 80) {
    TarnAnimation(boss, "walk");
    const irányX = t.dx > 0 ? 1 : -1;
    boss.move(irányX * boss.speed, 0);
    return;
}
```

Tarn legyőzése után a játékos megkapja a dupla ugrás képességet, és ezt a mentésben is eltároljuk:

```
mentett.bosses.Tarn = true;
mentett.ability_unlocked.double_jump = true
```

Összességében a két boss ugyanarra az alapelvre épül, mint a scraplet: figyelik a játékos helyzetét, állapot alapján döntenek, majd támadást vagy mozgást hajtanak végre. A különbség az, hogy a bossok több támadással, arénakezeléssel, mentési következményekkel és jutalmakkal rendelkeznek, ezért sokkal fontosabb szerepük van a játék előrehaladásában.

A játékos létrehozása a `jatekos_betoIt()` függvényben történik. Itt kapja meg a Kaboom komponenseit, például a sprite-ot, pozíciót, ütközési területet és fizikát.

```
const player = k.add([
  k.sprite("player"),
  k.pos(xpos, ypos - 30),
  k.anchor("center"),
  k.area({
    shape: new k.Rect(k.vec2(0, 1), 20, 30),
  }),
  k.body(),
  "player",
]);
```

A `ypos - 30` azért szükséges, mert így a játékos nem a talpánál, hanem a testének középpontjához igazítva kerül a pályára. Ez pontosabb spawnolást és ütközéskezelést ad.

A játékos életereje a mentési adatok alapján áll be. Alapból 5 szívvel indul, de ha a játékos korábban felvette a bónusz HP-kat, akkor több szívvel kezd:

```
let kezdoSzivek = 5;

if (aktivMentesAdatok?.world_interactions?.["bonus-hp-1"] === true) {
  kezdoSzivek += 1;
}

if (aktivMentesAdatok?.world_interactions?.["bonus-hp-2"] === true) {
  kezdoSzivek += 1;
}

if (aktivMentesAdatok?.world_interactions?.["bonus-hp-3"] === true) {
  kezdoSzivek += 1;
}
```

Ezután a HP rendszer és a HP UI is létrejön:

```
hpRendszerBeallitas(player, kezdoSzivek);
player.hpUI = hpUI(k, player);
```

A játékos több állapotváltozóval rendelkezik. Ezek alapján dönti el a program, hogy éppen mozoghat-e, támadhat-e, sérül-e, létrán van-e vagy menüben van-e.

```
player.letaranVan = false;
player.aktivLetra = null;
player.letraSebesseg = 100;
player.tamad = false;
player.tamadasAblakNyitva = false;
player.serul = false;
player.knockbackX = 0;
player.knockbackY = 0;
player.knockbackTimer = 0;
player.menuNyitva = false;
player.menu2Nyitva = false;
player.slashCooldown = 0;
player.slashCooldownMax = 15;
player.dashSpeed = 520;
player.dashTime = 0.16;
```

A támadásnál a játékos rövid ideig létrehoz egy láthatatlan hitboxot. Ha ez hozzáér egy ellenséghez vagy bosshoz, akkor az ellenfél sebződik.

```
const attackHitbox = k.add([  
  
    k.pos(  
  
        player.flipX ? player.pos.x + 18 : player.pos.x - 38,  
  
        player.pos.y - 10  
  
    ),  
  
    k.rect(40, 35),  
  
    k.area(),  
  
    k.opacity(0),  
  
    "player_attack_hitbox",  
  
]);
```

A játékos később különleges képességeket is használhat. A dash csak akkor működik, ha a mentésben fel van oldva:

```
const dashUnlocked =  
    aktivMentesAdatok?.data?.mentett_adatok?.ability_unlocked?.dash  
=== true;  
  
if (!dashUnlocked) return;
```

A dupla ugrás is ugyanígy a mentésből kerül ellenőrzésre:

```
function doubleJumpFeloldva() {  
    return (  
  
    aktivMentesAdatok?.data?.mentett_adatok?.ability_unlocked?.double_jump  
=== true  
    );  
}
```


A HP kezelő külön fájlban van, hogy az életerő logikája ne keveredjen közvetlenül a játékos mozgásával. A `hpRendszerBeallitas()` beállítja a maximális és aktuális HP-t:

```
let plussz_hp_az_alaphoz=0;

for(let i=0; i<3; i++){

if(aktivMentesAdatok.data.mentett_adatok.world_interactions[`bonus-hp-${i+1}`]){
    plussz_hp_az_alaphoz++;
}
}

player.maxHp = (kezdoSzivek + plussz_hp_az_alaphoz) * 2;

const mentettHp = localStorage.getItem("player_current_hp");

if (mentettHp !== null) {
    player.hp = Number(mentettHp);
} else {
    player.hp = player.maxHp;
}
```

A sebzés a `sebzesAdas()` függvénnyel történik. Ez csökkenti a játékos HP-ját, frissíti a UI-t, elmenti az aktuális HP-t, majd rövid ideig sebezhetetlenné teszi a játékost.

```
player.hp -= mennyiseg;

if (player.hp < 0) {
    player.hp = 0;
}

localStorage.setItem("player_current_hp", player.hp);

if (player.hpUI) {
    player.hpUI.frissit();
}

console.log(`Player HP: ${player.hp}/${player.maxHp}`);

player.invulnerable = true;

player.serul = true;
```

Sebzés után a játékos kap egy kis knockbacket is, hogy érezhető legyen a találat:

```
const irany = player.flipX ? -1 : 1;
player.knockbackX = irany * 440;
player.knockbackY = -120;
player.knockbackTimer = 0.18;
```

A gyógyítás egyszerűen visszatölti a HP-t, de nem engedi a maximum fölé menni:

```
player.hp += mennyiseg;

if (player.hp > player.maxHp) {
    player.hp = player.maxHp;
}

localStorage.setItem("player_current_hp", player.hp);

if (player.hpUI) {
    player.hpUI.frissit();
}
```

Halál esetén a játékos animációt vált, törlődik az ideiglenesen mentett HP, majd a program visszatölti a legutóbbi mentési pontot:

```
localStorage.removeItem("player_current_hp");

player.tamad = false;

if (player.vel) {
  player.vel.x = 0;
  player.vel.y = 0;
}

player.play("die");

k.wait(1.2, () => {
  if (player.exists()) {
    player.destroy();
  }

  const savepoint =
    localStorage.getItem("last_loaded_savepoint") || "kezdomap_1";

  respawnSavepointAlapjan(k, savepoint);
});
```

Összességében a játékos rendszere több részből áll: mozgás, támadás, interakció, képességek, menükezelés és HP kezelés. A HP kezelő külön fájlba került, mert így tisztábban kezelhető a sebzés, gyógyítás, maximális életerő növelése és a halál utáni újratöltés.

A **Szobakezelo.js** feladata, hogy a pályákhoz kapcsolódó közös logikákat egy helyen kezelje. Ide tartozik például a háttér beállítása, a colliderek létrehozása, az NPC-k elhelyezése, a szobák közötti átjárás, a mentéspontok, a törhető falak, a létrák és a pályához tartozó effektek kezelése.

A háttérszín beállítása egy egyszerű függvénnyel történik. Ez egy teljes képernyős, fix hátteret hoz létre a megadott színnel:

```
export function setBackgroundColor(k, hexColorCode) {
  k.add([
    k.rect(k.width(), k.height()),
    k.color(k.Color.fromHex(hexColorCode)),
    k.fixed(),
  ]);
}
```

A pályák ütközési felületeit a `MapColliderek()` függvény hozza létre. Ez a Tiled-ből érkező objektumok alapján láthatatlan falakat és talajokat generál. Kezeli a sima téglalap alakú collidereket és a polygon típusú formákat is.

```
export function MapColliderek(k, map, colliderek, forcedTag = null) {
  for (const collider of colliderek) {
    const objektumTag = forcedTag || collider.type || "Solid";

    if (collider.polygon) {
      const coordinates = [];
      for (const point of collider.polygon) {
        coordinates.push(k.vec2(point.x, point.y));
      }

      map.add([
        k.pos(collider.x, collider.y),
        k.area({
          shape: new k.Polygon(coordinates),
          collisionIgnore: ["Solid"],
        }),
        k.body({ isStatic: true }),
        "Solid",
        objektumTag,
      ]);
      continue;
    }

    map.add([
      k.pos(collider.x, collider.y),
      k.area({
```

```

        shape: new k.Rect(k.vec2(0), collider.width, collider.height),
        collisionIgnore: ["Solid"],
    )),
    k.body({ isStatic: true }),
    "Solid",
    objektumTag,
  ]));
}
}

```

A `SzobavaltozatoKezelo()` a szobák közötti átjárásért felel. Létrehoz egy láthatatlan átjáró hitboxot, és ha a játékos hozzáér, akkor a program átvált a cél szobára.

```

export function SzobavaltozatoKezelo(
  k,
  atjaroX,
  atjaroY,
  atjaroWidth,
  atjaroHeight,
  celSzoba,
  belepesiPont = null,
  atjaroTag = "atjaro"
) {
  //átjáró hitbox
  k.add([
    k.pos(atjaroX, atjaroY),
    k.rect(atjaroWidth, atjaroHeight),
    k.area(),
    k.opacity(0),
    atjaroTag,
  ]);

  //egyedi tag
  k.onCollide("player", atjaroTag, async () => {
    //console.log("Átjáró aktiválva:", atjaroTag);
    const container = document.querySelector("body");
    container.className = '';
    const kodkezelo = await Kod();
    kodkezelo.stop();
    k.go(celSzoba, { szoba_belepesi_pont: belepesiPont });
  });
}

```

```
});
}
```

A mentés pontokat a `MentesCollider()` kezeli. Ez egy láthatatlan objektumot hoz létre a pályán, amely eltárolja a hozzá tartozó mentési pont nevét.

```
export function MentesCollider(k, colliderObj, savepointNev) {
  const mentesPont = k.add([
    k.pos(colliderObj.x, colliderObj.y),
    k.area({
      shape: new k.Rect(k.vec2(0), colliderObj.width,
colliderObj.height),
    }),
    k.opacity(0),
    "mentespont",
  ]);

  mentesPont.savepointNev = savepointNev;
  console.log("Mentéspont objektum: ", mentesPont);

  return mentesPont;
}
```

A teljes mentés a `TeljesMentesLetrehozo()` függvényben történik. Ha a játékos be van jelentkezve, akkor az adatokat az API-n keresztül az adatbázisba mentjük. Ha vendégként játszik, akkor a mentés a `localStorage`-ba kerül.

```
let mentendo_adatok = mentett_adatok.data.mentett_adatok;

if (user_id != 0) {
  console.log("Mentendő adatok: ", mentendo_adatok);
  const response = await
fetchData("http://127.0.0.1:3000/api/mentes/update-teljes-mentes",
"PATCH", {
  user_id: user_id,
  mentes_id: mentes_id,
  mentett_adatok: mentendo_adatok
});
}
```

```

let Unlocked_achis=[];

if(aktivMentesAdatok.data.mentett_adatok.NPC_interactions.Prowl){
    Unlocked_achis.push(1);
}

if(aktivMentesAdatok.data.mentett_adatok.NPC_interactions.Ratchet){
    Unlocked_achis.push(2);
}

if(aktivMentesAdatok.data.mentett_adatok.NPC_interactions.Swindle){
    Unlocked_achis.push(3);
}

if(aktivMentesAdatok.data.mentett_adatok.NPC_interactions.Tailgate){
    Unlocked_achis.push(8);
}

if(aktivMentesAdatok.data.mentett_adatok.NPC_interactions.Chromedome_and_Ratchet){
    Unlocked_achis.push(9);
}

if(aktivMentesAdatok.data.mentett_adatok.world_interactions["crystal-heart-lava-protection"]){
    Unlocked_achis.push(6);
}

if(aktivMentesAdatok.data.mentett_adatok.world_interactions["lighthouse-sea-of-flowers-cutscenes"]){
    Unlocked_achis.push(4);
}

if(aktivMentesAdatok.data.mentett_adatok.bosses.Tarn){
    Unlocked_achis.push(7);
}

```

```

        if(aktivMentesAdatok.data.mentett_adatok.bosses.Sparkeater){
            Unlocked_achis.push(5);
        }

        for(const element of Unlocked_achis){
            await fetchData("http://127.0.0.1:3000/api/updateachivements",
"PATCH", {
                user_id: user_id,
                achivement_id: element
            });
        }

        const data = await response;
        return data;
    } else {
        let mentesneve = "mentes_" + mentes_id;
        localStorage.setItem(mentesneve,
            JSON.stringify({
                success: true,
                data: { mentett_adatok: mentendo_adatok }
            }),
        );

        const achi_en = localStorage.getItem('achivements_en');
        const achi_hu = localStorage.getItem('achivements_hu');

        const achi_en_data=achi_en ? JSON.parse(achi_en) : {};
        const achi_hu_data=achi_hu ? JSON.parse(achi_hu) : {};

        if(aktivMentesAdatok.data.mentett_adatok.NPC_interactions.Prowl){
            achi_en_data.data[0].unlocked=1;
            achi_hu_data.data[0].unlocked=1;
        }

        if(aktivMentesAdatok.data.mentett_adatok.NPC_interactions.Ratchet){
            achi_en_data.data[1].unlocked=1;
            achi_hu_data.data[1].unlocked=1;
        }
    }

```



```

if(aktivMentesAdatok.data.mentett_adatok.NPC_interactions.Swindle){
    achi_en_data.data[2].unlocked=1;
    achi_hu_data.data[2].unlocked=1;
}

if(aktivMentesAdatok.data.mentett_adatok.NPC_interactions.Tailgate){
    achi_en_data.data[7].unlocked=1;
    achi_hu_data.data[7].unlocked=1;
}

if(aktivMentesAdatok.data.mentett_adatok.NPC_interactions.Chromedome_and_Ratchet){
    achi_en_data.data[8].unlocked=1;
    achi_hu_data.data[8].unlocked=1;
}

if(aktivMentesAdatok.data.mentett_adatok.world_interactions["crystal-heart-lava-protection"]){
    achi_en_data.data[5].unlocked=1;
    achi_hu_data.data[5].unlocked=1;
}

if(aktivMentesAdatok.data.mentett_adatok.world_interactions["lighthouse-sea-of-flowers-cutscenes"]){
    achi_en_data.data[3].unlocked=1;
    achi_hu_data.data[3].unlocked=1;
}

if(aktivMentesAdatok.data.mentett_adatok.bosses.Tarn){
    achi_en_data.data[6].unlocked=1;
    achi_hu_data.data[6].unlocked=1;
}

if(aktivMentesAdatok.data.mentett_adatok.bosses.Sparkeater){
    achi_en_data.data[4].unlocked=1;
    achi_hu_data.data[4].unlocked=1;
}

```

```

const updated_achi_en_data = JSON.stringify(achi_en_data);
const updated_achi_hu_data = JSON.stringify(achi_hu_data);

localStorage.setItem("achivements_en", updated_achi_en_data);
localStorage.setItem("achivements_hu", updated_achi_hu_data);
}

```

A szoba kezelő nem csak mentést kezel, hanem a pályaelemek állapotát is. Például a **BreakableFal()** törhető falat hoz létre, amelynek saját HP-ja van. Ha a játékos megtámadja, csökken az élete, majd ha elfogy, a fal eltűnik és a mentésben is eltárolódik, hogy már el lett törve.

```

k.onCollide("player_attack_hitbox", tag, (hitbox, wall) => {
  if (torott) return;
  if (!serulhet) return;

  serulhet = false;
  currentHp--;

  console.log(`${tag} megütve. Maradék HP: ${currentHp}`);

  if (remegjen && falSprite) {
    let razasIdo = 0.18;

    const razas = k.onUpdate(() => {
      if (!falSprite.exists()) {
        razas.cancel();
        return;
      }

      falSprite.pos.x = falSprite.originalX + (Math.random() * 8 -
4);

      razasIdo -= k.dt();
      if (razasIdo <= 0) {
        falSprite.pos.x = falSprite.originalX;
        razas.cancel();
      }
    });
  }
});
}

```

```

    if (currentHp <= 0) {
        torott = true;

        if (saveKey && aktivMentesAdatok) {
            aktivMentesAdatok.data.mentett_adatok.world_interactions[saveKey] =
            true;
        }

        if (falSprite && falSprite.exists()) {
            falSprite.destroy();
        }

        if (wall.exists()) {
            wall.destroy();
        }

        if (hitbox.exists()) {
            hitbox.destroy();
        }

        return;
    }

    k.wait(0.2, () => {
        serulhet = true;
    });
});

```

A `LetraCollider()` a létrák működését kezeli. Ha a játékos a létra közelében van, akkor interakcióval fel tud mászni rá, a program pedig eltárolja, hogy éppen létrán van.

```
k.onKeyPress(interactKey, () => {
    if (player.aktivLetra === letra && !player.letaranVan) {
        player.letaranVan = true;

        if (player.tutorial) {
            player.tutorial.markDone("ladder_interact");

            player.tutorial.showOnce(
                "ladder_climb",
                "W / S - mászás"
            );
        }

        player.pos.x = letra.pos.x + letra.letraWidth / 2;

        if (player.vel) {
            player.vel.x = 0;
            player.vel.y = 0;
        }
    }
});
```

A `CollapsingPlatform()` olyan platformot hoz létre, amely először remeg, majd néhány másodperc után eltűnik, később pedig újra megjelenik. Ez mozgásra és gyors döntésre kényszeríti a játékost.

```
p.onUpdate(() => {
    if (collapsed) return;

    if (playerOn) {
        timer += k.dt();

        //3 mp után remegjen
        if (timer >= 3 && timer < 5) {
            shaking = true;
        }

        if (shaking && spriteObj) {
            spriteObj.pos.x = spriteObj.originalX + (Math.random() * 4 - 2);
            spriteObj.pos.y = spriteObj.originalY + (Math.random() * 4 - 2);
        }

        //5 mp után tűnjön el
        if (timer >= 5) {
            collapsed = true;
            playerOn = false;
            shaking = false;
            timer = 0;

            if (spriteObj) {
                spriteObj.opacity = 0;
                spriteObj.pos.x = spriteObj.originalX;
                spriteObj.pos.y = spriteObj.originalY;
            }

            p.destroy();

            k.wait(3, () => {
                collapsed = false;

                if (spriteObj) {
                    spriteObj.opacity = 1;
                }
            });
        }
    }
});
```

```

    }

    platform = letrehozCollider();
  });
}
}
});

```

A szoba kezelő tartalmaz látvány effekteket is. Ilyen például az eső, a köd és a hamu. Ezek külön `start()` és `stop()` függvényekkel indíthatók és állíthatók le, hogy szoba váltáskor ne maradjanak bent felesleges effektek.

```

export function EffektTorles() {
  if (window.kodkezelő) {
    window.kodkezelő.stop();
    window.kodkezelő = null;
  }

  if (window.hamukezelő) {
    window.hamukezelő.stop();
    window.hamukezelő = null;
  }

  if (window.esokezelő) {
    window.esokezelő.stop();
    window.esokezelő = null;
  }
}

```

A szobák zenéjét a `szoba_zene_beallitas()` függvény állítja át. Ez megváltoztatja a háttérzene forrását, majd újratölti és elindítja azt.

```
export function szoba_zene_beallitas(szoba_neve) {  
  let zenedoboza = document.getElementById("zenemarad");  
  let zene = "../audio/" + szoba_neve + ".mp3";  
  
  zenedoboza.src = zene;  
  zenedoboza.load();  
  
  if (zenedoboza.muted) {  
    zenedoboza.muted = false;  
    zenedoboza.play();  
  }  
}
```

Összességében a `Szobakezelo.js` a játék egyik központi segéd fájlja. Nem egy konkrét szobát ír le, hanem azokat az újrahasználató rendszereket tartalmazza, amelyekre több pályának is szüksége van: colliderek, átjárók, mentés pontok, létrák, törhető falak, beszakadó platformok, effektek, achievement mentés és zene váltás.

A játék szobái külön JavaScript fájlokban vannak kezelve. Ezek közül az egyik jó példa az `Iacon`, mert több fontos rendszer is megtalálható benne: pályabetöltés, játékos spawn, mentéspont, NPC, scrapletek, létrák, törhető falak, beszakadó talaj és több szobába vezető átjáró.

A szoba indításakor először töröljük az előző speciális effekteket, beállítjuk az adott szoba zenéjét és a háttérszint:

```
EffektTorles();

szoba_zene_beallitas("Iacon_room");

setBackgroundColor(k, "#000000");
```

Ezután a program lekéri az Iaconhoz tartozó Tiled map JSON fájlt az API-ból:

```
const mapData = await fetchData(
  "http://127.0.0.1:3000/api/map_data/Iacon_city.json",
);
```

A szoba mérete fixen meg van adva. Itt egy 60×50 tile-os pályáról van szó, ahol egy tile 32 pixel:

```
const mapW = 32 * 60;
const mapH = 32 * 50;
```

A játékos kezdő pozíciója alapból a mentéspont layerből jön, de ha másik szobából érkezik, akkor a **szoba_belepesi_pont** alapján más spawn pontot kap:

```
if (szoba_belepesi_pont != null) {
  switch (szoba_belepesi_pont) {
    case "Back_from_Mitteous_1_and_Medical_Bay":
      xpos = mapData.data.layers[28].objects[0].x;
      ypos = mapData.data.layers[28].objects[0].y;
      break;

    case "Back_from_Mitteous_2":
      xpos = mapData.data.layers[29].objects[0].x;
      ypos = mapData.data.layers[29].objects[0].y;
      break;

    case "Back_from_Mitteous_3":
      xpos = mapData.data.layers[30].objects[0].x;
      ypos = mapData.data.layers[30].objects[0].y;
```



```

        break;

    case "Back_From_Kaon_1":
        xpos = mapData.data.layers[21].objects[0].x;
        ypos = mapData.data.layers[21].objects[0].y;
        break;

    case "Back_From_Kaon_2":
        xpos = mapData.data.layers[22].objects[0].x;
        ypos = mapData.data.layers[22].objects[0].y;
        break;

    case "Back_from_crystal_city":
        xpos = mapData.data.layers[14].objects[0].x;
        ypos = mapData.data.layers[14].objects[0].y;
        break;

    case "Falling_from_Mitteous_4":
        xpos = mapData.data.layers[23].objects[0].x;
        ypos = mapData.data.layers[23].objects[0].y;
        break;

    case "savepoint_3":
        xpos = mapData.data.layers[9].objects[0].x;
        ypos = mapData.data.layers[9].objects[0].y;
        break;

    }
}

```

Ez azért fontos, mert egy Metroidvania játékban ugyanabba a szobába több irányból is be lehet lépni, és nem mindegy, hogy a játékos hol jelenik meg.

A pálya fő képe és colliderei ezután jönnek létre:

```

const map = k.add([k.pos(0, 0), k.sprite("Iacon")]);

MapColliderek(k, map, szoba_layerek[12].objects);

```

A mentési adatok alapján a program eldönti, hogy bizonyos pályaelemeknek meg kell-e jelenniük. Például ha egy falat a játékos már korábban széttört, akkor azt nem hozzuk létre újra:

```
const wall1Broken =
aktivMentesAdatok?.data?.mentett_adatok?.world_interactions?.["Iacon_br
eakable-wall1"] ;
const wall2Broken =
aktivMentesAdatok?.data?.mentett_adatok?.world_interactions?.["Iacon_br
eakable-wall2"] ;
```

```
let breakableWall1 = null;
  if (!wall1Broken) {
    breakableWall1 = k.add([k.pos(0, 0),
k.sprite("Iacon_Breakable_wall_1")]);
  }

  let breakableWall2 = null;
  if (!wall2Broken) {
    breakableWall2 = k.add([k.pos(0, 0),
k.sprite("Iacon_Breakable_wall_2")]);
  }
```

A törhető falhoz külön collider is készül. Ha a játékos elégszer megüti, a fal eltűnik, és a mentésbe bekerül, hogy már el lett törve:

```
if (!wall1Broken && breakableWall1Layer?.objects?.[0] &&
breakableWall1) {
  BreakableFal(
    k,
    breakableWall1Layer.objects[0],
    breakableWall1,
    2,
    true,
    "breakable_wall_1",
    "Iacon_breakable-wall1"
  );
}
```

Iaconban beszakadó talaj is található. Amikor a játékos rálép az adott trigger zónára, a talaj eltűnik, és ez is bekerül a `world_interactions` adatok közé:

```
k.onCollide("player", "collapsing_ground_trigger_1", () => {
    if (collapseTriggered1) return;
    collapseTriggered1 = true;

    console.log("Beomló talaj 1 aktiválva");

    if (aktivMentesAdatok) {
        aktivMentesAdatok.data.mentett_adatok.world_interactions["Iacon_breakable-ground1"] = true;
    }

    if (collapsingGround1) {
        collapsingGround1.destroy();
    }
    collapsingGroundTrigger1.destroy();
});
```

A mentéspont szintén a Tiled objektum layeréből kerül beolvasásra:

```
const savepointObj = mapData.data.layers[9].objects[0];
const savepointNev = mapData.data.layers[9].name;
MentesCollider(k, savepointObj, savepointNev);
```

Ha Swindle NPC-vel még nem beszélt a játékos, akkor az NPC is létrejön:

```
let kelleswindle =
!aktivMentesAdatok.data.mentett_adatok.NPC_interactions.Swindle;

if (kelleswindle) {
    NPCCollider(k, szoba_layerek[32].objects, "Swindle");
}
```

Ezután betöltjük magát a játékost, majd hozzáadjuk a kamerakezelést:

```
const player = await jatekos_betolt(k, xpos, ypos, "Iacon");
```

```
Kamera_kezelo(k, xpos, ypos, player, mapW, mapH);
```

A scrapletek spawn pontjai szintén a map JSON-ból jönnek. A program végigmegy a spawn objektumokon, és mindegyikhez létrehoz egy scrapletet:

```
if (scrapletSpawnLayer && scrapletSpawnLayer.objects) {  
    scrapletSpawnLayer.objects.forEach((obj) => {  
        ScrapletLetrehoz(k, obj.x, obj.y, player, 100);  
    });  
}
```

A létrák hasonló módon jönnek létre:

```
if (ladderLayer && ladderLayer.objects) {  
    ladderLayer.objects.forEach((obj) => {  
        LetraCollider(k, obj, player, settings.controls.interact,  
settings.controls.jump);  
    });  
}
```

A szoba végén jönnek létre az átjárók. Ezek láthatatlan hitboxok, amelyek másik szobába viszik a játékost:

```
SzobavaltozatoKezelo(  
    k,  
    mapData.data.layers[24].objects[0].x,  
    mapData.data.layers[24].objects[0].y,  
    mapData.data.layers[24].objects[0].width,  
    mapData.data.layers[24].objects[0].height,  
    "Medbay",  
    "Back_from_Iacon",  
    "atjaro_medbay"  
);
```

Iacon különösen jó példa a szobák működésére, mert sok másik helyszínhez kapcsolódik: Medbay, Mitteous Plateau, Kaon, Crystal City és a Leesős hely is innen érhető el. Emiatt ez a szoba jól bemutatja, hogyan működik a nemlineáris pályaszerkezet.

Összességében egy map fájl fő feladata az, hogy betöltse a Tiled-ből érkező pálya adatokat, létrehozza a szükséges játék objektumokat, beállítsa a játékos helyzetét, kezelje az adott szobához tartozó interakciókat, majd összekösse a szobát a többi pályarésszel.

A kamera kezelését a `Kamera_kezelo()` függvény végzi. Ennek a célja, hogy a kamera kövesse a játékost, de ne hirtelen, rángatózva mozogjon, hanem finoman késleltetve.

A kamera induláskor a játékos kezdő pozíciójára áll, majd nagyítást kap:

```
k.camPos(xpos, ypos - 30);  
k.camScale(4);
```

A kamera nem közvetlenül a játékosra van ráállítva, hanem egy láthatatlan `cameraTarget` objektumot követ. Ez azért jobb, mert így a kamera mozgását lehet simítani:

```
const cameraTarget = k.add([  
  k.pos(xpos, ypos - 30),  
]);
```

Mozgás közben a kamera kicsit előre néz abba az irányba, amerre a játékos halad. Ezt a `lookAhead` változó kezeli:

```
if (k.isKeyDown(settings.controls.forward)) {  
  targetLook = maxLook;  
} else if (k.isKeyDown(settings.controls.back)) {  
  targetLook = -maxLook;  
}  
  
lookAhead += (targetLook - lookAhead) * lookSpeed;
```

A függvény a függőleges mozgást is figyeli. Ha a játékos ugrik, a kamera kicsit felfelé néz, ha pedig esik, akkor inkább lefelé, hogy a játékos lássa, hova fog érkezni:

```
if (!player.isGrounded() && velocityY < -2) {  
    targetVertical = -maxUpLook;  
}  
  
if (!player.isGrounded() && velocityY > 3) {  
    targetVertical = maxDownLook;  
}  
  
if (player.isGrounded()) {  
    targetVertical = 0;  
}
```

A kamera simított követéssel mozog a célpont felé, ezért nem ugrik azonnal a játékos pozíciójára:

```
cameraTarget.pos.x += (targetX - cameraTarget.pos.x) * followSpeed;  
cameraTarget.pos.y += (targetY - cameraTarget.pos.y) * followSpeed;
```

Fontos része a kamerának a pálya határok kezelése. Ez akadályozza meg, hogy a kamera kilásson a pályán kívülre:

```
camX = Math.max(halfW, camX);  
camX = Math.min(mapW - halfW, camX);  
  
camY = Math.max(halfH, camY);  
camY = Math.min(mapH - halfH, camY);
```

Boss aréna esetén a kamera külön korlátozást kap. Ilyenkor nem az egész pályán mozoghat, hanem csak az aréna méretei között:

```
if (bossArenaAktiv && bossArenaObj) {  
    const arenaMinX = bossArenaObj.x + halfW;  
    const arenaMaxX = bossArenaObj.x + bossArenaObj.width - halfW;  
    const arenaMinY = bossArenaObj.y + halfH;  
    const arenaMaxY = bossArenaObj.y + bossArenaObj.height - halfH;  
  
    if (arenaMaxX >= arenaMinX) {  
        camX = Math.max(arenaMinX, camX);  
        camX = Math.min(arenaMaxX, camX);  
    } else {  
        camX = bossArenaObj.x + bossArenaObj.width / 2;  
    }  
  
    if (arenaMaxY >= arenaMinY) {  
        camY = Math.max(arenaMinY, camY);  
        camY = Math.min(arenaMaxY, camY);  
    } else {  
        camY = bossArenaObj.y + bossArenaObj.height / 2;  
    }  
}
```

A végén a kiszámolt kamera pozíció beállításra kerül:

```
k.camPos(camX, camY);
```

Összességében a kamera nem egyszerűen csak a játékos pozícióját követi, hanem figyeli a mozgás irányát, az ugrást, az esést, a pálya határokat és a boss arénákat is. Emiatt a játékos jobban látja, merre halad, miközben a kamera mozgása természetesebb és kevésbé zavaró.

A játék felépítése, logikai háttere, története és érdekességek

A játékon belüli részt több részre (ettől a ponttól szobákra) felosztottuk. Ezzel a játék logikát több blokkra fel tudjuk osztani, ezzel létrehoztuk a nem lineáris játék lehetőségét.

Az intronak köszönhetően megismerjük a játék alap kontextusát és megismerjük a játék célját. Eljutni a Lost Light hajóhoz, hogy utazhassunk a többiekkel az űrben.

A kezdő szoba, ahol az intro után kötünk ki, Prowlnak (az egyik NPC-nek) az irodája. Itt először meglátjuk a karakterünket a cicát (, akit a Rewind nevezetű karakter alapján csináltam, mert nem tudok robotokat jól animálni :,c). Ebben az irodában az alapvető mozgást megismerjük (baloldalt lehet ugráltatni, hogy tapasztaljuk a gravitációt), a létrával való interakciót és felmászást és az NPC-vel való interakciót. Ennek a szobának a jobboldali végén van egy átjáró, ami a következő szobába átvezet minket.

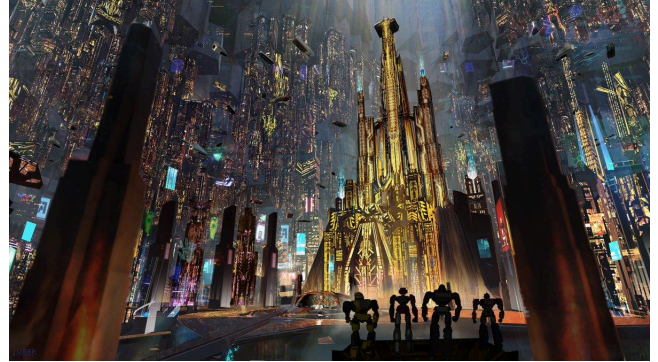
A Mitteous fennsík. A fennsík maga az IDW kiadó ház kitalálmánya. A fennsíkon található az első mentési pontja a játéknak. Ebben a szobában már a játék nem linearitása jelen van több szempontból:

1. A beszakadó talaj, amin alapvetően nem tudunk átjutni és vissza kell térni később egy olyan képességgel amivel át tudunk jutni.
2. Alapvetően ott 1 NPC interakció esemény van, aminek 2 előfeltétele van:
 1. Találkozunk Ratchettel a "Kórházban"
 2. Találkozunk Tailgattal a "Leesős szobában"

A fennsík több ponton keresztül van 3 szobával összekötve:

1. Prowl irodája
2. A Lost Light-t tartalmazó szoba
3. Iacon városa

lacon városa a következő szoba, ahova minden esetben eljutunk. A Mitteous fennsíkról ide leesni lehet csak és csak 3 ponton lehet visszamenni a fennsíkra a 4-ből. lacon a transformers minden egyes iterációjában jelen levő város, amely jellemzően az Autobotok által ellenőrzött város. A játékhoz a 2024-ben kiadott Transformers: One filmben található földalatti lacont használjuk alapul. A játékunkban ennek a szobának az a szerepe, hogy a játékos megismerkedjen a falak törésével, rejtett szobákkal és egyszerű ellenségek elleni harccal. A szobában található ellenségek a “scraplet” nevezetű lények, akiknek az



3.ábra: lacon városa

alapját a Transformers: Prime (2010) -es sorozatából vettük ki, de mindegyik transformers szériában újra és újra felhozott téma a szimpla koncepciójuk miatt. Csak képzeljük el, hogy milyen lenne, ha egy faj specifikusan azért létezne, hogy élve megegyenek minket? Ez emberként nem létező probléma szerencsére.



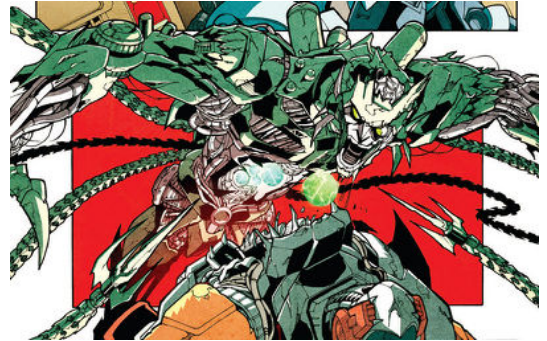
4.ábra: Scraplet

A szobában az egyik rejtett fal mögött található egy rejtett szobarész, ahol Swindle-lel találkozhatunk és van egy mentéspont. A szobában ezen kívül a rejtett szobarész mellett található egy beszakadó talaj, ami 2 szoba átjáróhoz (Crystal városához és Kaon városához) és egy törhető falhoz vezet. A fal törése után egy másik beszakadó talajhoz vezet. Ehhez a beszakadó talajhoz úgy is el lehet jutni, hogy a szoba jobb részére ugrunk és ott maradunk, vagy a Mitteous fennsíkon beszakadó talaj után nem tudunk időben landolni. Ezekben az esetekben átesünk a “leesős hely” nevű szobába.

Ezenek a szobákon kívül még kettő másik szobával vannak összekötve. A kórházzal és Kaon városával.

A kórházi szoba egy egyszerű pálya logikailag, találkozunk Ratchettel (NPC) és van egy mentéspont. Az itteni interakció az egyik előfeltétele a fenssíki jelenetnek. Ez a szoba csak Iacon városával van összekötve logikailag.

A kórház alatti pálya Kaon városa. A Transformers sorozatokban ez jellemzően a Deceptikonok központja, ezért az első boss, a Sparkeater (akivel remélhetőleg hamarabb találkozunk, mint a másodikkal) ebbe a szobába pakoltuk. Amikor legyőztük, akkor egy kulcsot és a dash képességet megkapjuk. A kulcsot a Crystal városban lehet felhasználni. Kaon, ezen kívül tele van scrapletekkel és egy mentésponttal, hogy ne unatkozzunk.



5.ábra: Sparkeater boss

A Sparkeater bosztól balra található egy arany szív. A szív a mentésen belül növeli a maximum HP-nkat. Ezzel a Sparkeater elleni harc egy picit könnyebb lesz.

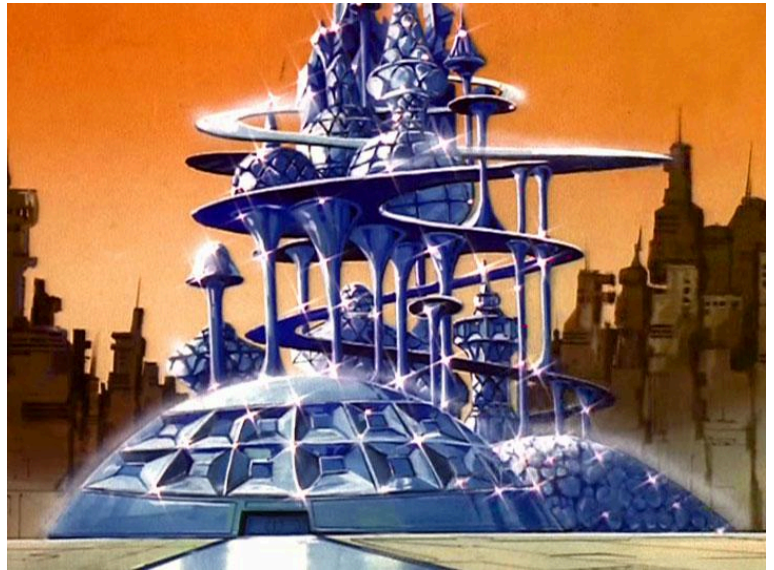
Kaonban több átjárási pont van. A következők ezek a pontokról fentről lefelé:

- Iacon (2 darab egymás után)
- Crystal város (1 darab)
- A temető (1 darab)
- Az olvasztó gödrök (2 darab egymás után)



6.ábra: Kaon városa

Crystal város, a sorozatok szerint mindig a legszebb város Cybertronon. Kristályokból épített épületek, utak és kertek. A játékban ez a szoba egy különleges szerepet tölt be. Abban az esetben, ha legyőzzük a Sparkeatert, kapunk egy kulcsot, amivel ebben a szobában ki tudjuk nyitni a zárt kaput. A kapu mögött található egy vörös szív, amit ha felveszünk, akkor az olvasztó gödrökben a láva nem fog minket halálra égetni. Viszont, érdemes a



7.ábra: Crystal city

lábunk alá figyelni, mert a kristályok közé be lehet esni és halálra tudnak minket szurkálni. Ez a szoba laconnal és Kaonnal van összekötve.

A temető egy kis szoba a Crystal város alatt és Kaonból lehet bejutni. A temetőben található egy világítótorny és maga a temető. A temetőnek a jelenete csak akkor érhető el, ha a világítótorny fel van kapcsolva. A temetős jelenet után elérhető lesz a játékos számára egy második támadás típust. Ez sokat segít a boss harcokban.

A "leesős szoba". Ebben a szobában Tailgate (NPC) és egy arany szív található. Tailgate a második előfeltétele a Mitteous fennsíkon található jelenetnek. Ezen kívül ennek a szobának 1 be- és 1 kijárata van. A bejárat lacon felől van, ahonnan ide beesünk és az egyetlen kijárat az olvasztó gödrökbe vezet. Ezzel a szobával effektíven a Mitteous fennsíkról egyből az olvasztó gödrökben köthetünk ki.

Az olvasztó gödrök. A szobában egy Tarn nevezetű boss található. A boss arénának van egy specialitása: a talaj be tud szakadni alattunk és bele tudunk esni a lávába. A lávának nem csak ez az egyetlen funkciója. Egy rejtett szobát is meg lehet találni a lávában, ahol a játék harmadik arany szívét találhatjuk. A játékban ezeken kívül nem található más arany szív. Az olvasztó gödrök a Transformers szériában az egyik szimbólum a háborúra.

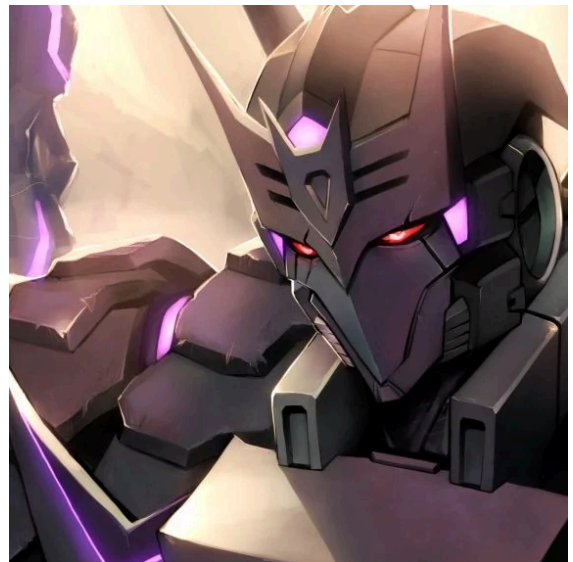


8.ábra: Az olvasztó gödrök

Az olvasztó gödrök a több milliós háborújuk alatt is még mindig ég. A foglyok, az ellenállók és az "újrahasznosított" fém testekből áll össze a hely.

A felsorolt szobák mind a játék folyamat részei. Egy szoba maradt hátra, ami lezárva ezt a játékot: A Lost Light hajót tartalmazó szoba. Itt, hogyha a fennsíki jelenetet végignéztük, akkor találkozhatunk Tailgattel, Ratchettel és Chromedomemmal. Velük itt már nem lehet beszélgetni. Viszont, a szobának a fő funkciója

a történet és a játék lezárása. Attól függően, hogy mennyire jártuk körbe a környezetet, 2 különböző vége lehet a történetnek. Az már másik kérdés, hogy mik azok a feltételek. ☹ (. ◡ ◡ .) ☹



9.ábra: Tarn



10.ábra: Lost Light hajó

Tesztelés leírása, folyamata és tesztdokumentáció

Egy játék tesztelése weboldalként nehéz. Nincsenek sajnos különleges könyvtárakra unit tesztek, úgyhogy a játék nagy részét felhasználói tesztekkel (pl.: az integrációs teszt) oldottuk meg. Viszont, nem csak egy játékunk van, hanem egy egyszerű felhasználóbarát nyitóoldalunk is van. Azokat a következő unit tesztekkel teszteltük:

Teszt száma	Teszt neve	Teszt leírása	Megjegyzés
1.	GET /api/test	A végpont működését teszteli	
2.	GET /api/testsql	Tesztadatot ad vissza	Szerver errorot kezel is
3.	GET /api/nyelv_alapjan_JSON_olvasas/:nyelv/:fajl	A main_menu.json magyar változatát adja vissza	400-as error, ha nem felel meg valami kezelése
4.	GET /api/felhasznalo/:id	A felhasználó beállításait adja vissza	null a return, ha nincs beállítás és 400-as error, ha hibás user ID kezelés
5.	GET /api/fiokadat/:id	A fiókadatot adja vissza	
6.	GET /api/showachivements/:id/:nyelv	Vissza kéri a jutalmakat a felhasználó számára	400-as error, ha érvénytelen paraméter kezelése

7.	GET /api/admin/users	A felhasználók adatait adja vissza (csak admin)	
8.	GET /api/mentesmeghiv /:user_id/:mentes_ id	A mentés adatait adja vissza	
9.	GET /api/map_data/:sz oba_neve	Visszakéri a kezdomap.json adatait	400-as error, ha nem JSON fájl kezelése
10.	GET /api/admin/elfelejte tt-jelszo-keresek	Jelszó visszaállítási kérések lekérdezése (csak adminoknak)	
11.	POST /api/register	Új felhasználó regisztrálása	409, ha létezik a felhasználó, 400-as error, ha érvénytelen az email formátum és A felhasználó adminisztrátori jogosultsággal rendelkezik (az apiba be van égetve 2 fix admin) kezelése

12.	POST /api/login	Sikeres bejelentkezés helyes hitelesítő adatokkal	401-es error, ha a felhasználónév nem létezik és 401-es error, ha hibás a jelszó
13.	POST /api/felhasznalo/beallitas	Hangerő frissítése, nyelv beállítás frissítése és kulcs kombinációk frissítése	400-as error, ha érvénytelen a user_id
14.	POST /api/kelleNPC	NPC interaction státuszának lekérdezése	
15.	POST /api/elfelejtett-jelszo	Jelszó-visszaállítási kérés létrehozása	400-as error, ha az email hiányzik és 404-es error, ha az email nem található kezelése
16.	PATCH /api/user/:id	Felhasználó adatainak frissítése	422-es error, ha hiányzik az adat kezelése
17.	PATCH /api/updateachievements	Frissíti a teljesítményt és elmenti az adatokat	
18.	PATCH /api/admin/reset-jelszo	Jelszó visszaállítása alaphelyzetbe	400-as error, ha hiányzik az adat kezelése

19.	PATCH /api/user/jelszo-csere/:id	Jelszó módosítása érvényes új jelszóval	400-as error, ha a jelszó nem felel meg a követelményeknek kezelése
20.	PATCH /api/mentes/update-savepoint	Frissíti a mentési pontot egy mentési slotot számára	400-as error, ha hiányzik az adat
21.	DELETE /api/user/:id	Törli a felhasználót	
22.	meghívja az oldalTakarito függvényt és lekéri a nyelvi adatokat	meghívja az oldalTakarito függvényt és lekéri a nyelvi adatokat	
23.	ha nincs #zenemarad, létrehozza a zene audio elemet a megfelelő beállításokkal	ha nincs #zenemarad, létrehozza a zene audio elemet a megfelelő beállításokkal	
24.	ha már létezik #zenemarad, nem hoz létre újat	ha már létezik #zenemarad, nem hoz létre újat	

25.	Zene tényleges be- és kikapcsolása	Zene tényleges be- és kikapcsolása	Mint kiderült, nincsen play() function a jest-ben, ezért egy beforeallban lekezelem a play()-t, hogy a teszt futhasson
26.	a hangerő változtatás eseményre frissíti a zene hangerejét	a hangerő változtatás eseményre frissíti a zene hangerejét	
27.	guest felhasználó esetén a "Bejelentkezés" gomb jelenik meg	guest felhasználó esetén a "Bejelentkezés" gomb jelenik meg	
28.	a bejelentkezés gombra kattintva megjelenik a modal	a bejelentkezés gombra kattintva megjelenik a modal	
29.	bejelentkezett felhasználó esetén a "Kijelentkezés" gomb jelenik meg	bejelentkezett felhasználó esetén a "Kijelentkezés" gomb jelenik meg	

30.	a jelentkezés gombra kattintva confirm jelenik meg, és ha igaz, guest lesz a felhasználó	a jelentkezés gombra kattintva confirm jelenik meg, és ha igaz, guest lesz a felhasználó	
31.	ha a confirm false, akkor nem történik jelentkezés	ha a confirm false, akkor nem történik jelentkezés	
32.	létrehozza a menü címet és gombokat, a kattintáseseménye ekkel	létrehozza a menü címet és gombokat, a kattintáseseménye ekkel	
33.	meghívja a sutiModalKeszit-et	meghívja a sutiModalKeszit-et	
34.	Alap JSON-t ad vissza sikeres fetch esetén	Alap JSON-t ad vissza sikeres fetch esetén	
35.	Ha van body, azt JSON-ként küldi	Ha van body, azt JSON-ként küldi	
36.	hiba a fetch hibás válaszával	hiba a fetch hibás válaszával	
37.	Hiba kiírása a: "Hiba: " résszel	Hiba kiírása a: "Hiba: " résszel	
38.	#zenemarad -on kívül mindent töröl	#zenemarad -on kívül mindent töröl	

39.	Nem szedi ki #zenemarad element-et	Nem szedi ki #zenemarad element-et	
40.	Átírja body.innerHTML-t "-re, ha #zenemarad nem létezik	Átírja body.innerHTML-t "-re, ha #zenemarad nem létezik	
41.	Vissza gombot hoz létre a megfelelő szöveggel, osztályokkal és ID-val	Vissza gombot hoz létre a megfelelő szöveggel, osztályokkal és ID-val	
42.	meghívja a createMainMenu függvényt, amikor rákattintanak	meghívja a createMainMenu függvényt, amikor rákattintanak	
43.	különböző gombokat hoz létre	különböző gombokat hoz létre	
44.	visszaad egy div elemet a dekor vonal osztállyal	visszaad egy div elemet a dekor vonal osztállyal	
45.	tartalmaz három span elemet a megfelelő osztályokkal és szöveggel	tartalmaz három span elemet a megfelelő osztályokkal és szöveggel	

46.	Nem csatolja automatikusan a DOM-hoz	Nem csatolja automatikusan a DOM-hoz	
47.	Minden hívásnál új elemet ad vissza	Minden hívásnál új elemet ad vissza	
48.	alapértelmezett értékek, ha nincs localStorage	alapértelmezett értékek, ha nincs localStorage	
49.	nyelv a localStorage-ból	nyelv a localStorage-ból	
50.	nyelv alapértelmezett 1, ha a localStorage üres string	nyelv alapértelmezett 1, ha a localStorage üres string	
51.	irNyelv() frissíti a nyelv beállítást	irNyelv() frissíti a nyelv beállítást	
52.	többszöri modulbetöltés után is működik az irNyelv	többszöri modulbetöltés után is működik az irNyelv	
53.	meghívja az oldalTakarito-t és lekéri a szükséges adatokat	meghívja az oldalTakarito-t és lekéri a szükséges adatokat	

54.	létrehoz 4 játékfájl elemet és megfelelő osztályokat ad nekik	létrehoz 4 játékfájl elemet és megfelelő osztályokat ad nekik	
55.	a játékfájl gombokra kattintva indítja a KaboomBetolto-t	a játékfájl gombokra kattintva indítja a KaboomBetolto-t	
56.	a visszaGomb hívása a megfelelő szöveggel, és hozzáadása a DOM-hoz	a visszaGomb hívása a megfelelő szöveggel, és hozzáadása a DOM-hoz	
57.	meghívja az oldalTakarito-t és lekéri a megfelelő JSON fájlt	meghívja az oldalTakarito-t és lekéri a megfelelő JSON fájlt	
58.	létrehozza a .credits_container- t és a címet	létrehozza a .credits_container- t és a címet	
59.	meghívja a dekor_vonal_blokk al-t	meghívja a dekor_vonal_blokk al-t	

60.	létrehozza a scrolldiv-et és benne a közreműködők listáit	létrehozza a scrolldiv-et és benne a közreműködők listáit	
61.	meghívja a visszaGombot a megfelelő szöveggel	meghívja a visszaGombot a megfelelő szöveggel	
62.	a vissza gomb bekerül a DOM-ba	a vissza gomb bekerül a DOM-ba	
63.	létrehozza a modalt és visszaadja a DOM elemet	létrehozza a modalt és visszaadja a DOM elemet	
64.	a bejelentkező űrlap mezői megfelelnek a nyelvi adatoknak'	a bejelentkező űrlap mezői megfelelnek a nyelvi adatoknak'	
65.	a regisztrációs űrlap mezői és jelszó követelmények megjelennek	a regisztrációs űrlap mezői és jelszó követelmények megjelennek	

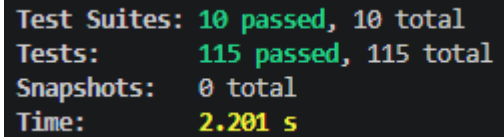
66.	bejelentkezés gomb sikeres válasz esetén localStorage-t ment és authChanged eseményt küld	bejelentkezés gomb sikeres válasz esetén localStorage-t ment és authChanged eseményt küld	
67.	sikertelen bejelentkezés esetén alert a hibaüzenettel	sikertelen bejelentkezés esetén alert a hibaüzenettel	
68.	elfelejtett jelszó modal megjelenik és eltűnik	elfelejtett jelszó modal megjelenik és eltűnik	
69.	kötelező jelszócsere modal megjelenik és működik	kötelező jelszócsere modal megjelenik és működik	
70.	a bezárás gomb elrejt a modalt és törli a mezőket	a bezárás gomb elrejt a modalt és törli a mezőket	
71.	vendég felhasználó esetén az alapértelmezett beállításokat használja	vendég felhasználó esetén az alapértelmezett beállításokat használja	

72.	bejelentkezett felhasználónál szerverről tölti be a beállításokat	bejelentkezett felhasználónál szerverről tölti be a beállításokat	
73.	meghívja az oldalTakarito-t és a fetchData-t	meghívja az oldalTakarito-t és a fetchData-t	
74.	megjeleníti a hangerő csúszkát és a billentyűket	megjeleníti a hangerő csúszkát és a billentyűket	
75.	a mentés gomb menti a beállításokat és alertet dob	a mentés gomb menti a beállításokat és alertet dob	
76.	admin joggal rendelkező felhasználónál megjelenik az Admin panel gomb	admin joggal rendelkező felhasználónál megjelenik az Admin panel gomb	
77.	Betölti az admin panelt	Betölti az admin panelt	
78.	oldaltakarito hívása és fordítások lekérése	oldaltakarito hívása és fordítások lekérése	

79.	adatok betöltése és tábla feltöltése	adatok betöltése és tábla feltöltése	
80.	Hiba esetén megjelenik a hibaüzenet	Hiba esetén megjelenik a hibaüzenet	
81.	Mentés és a PATCH kérés	Mentés és a PATCH kérés	
82.	Törlés és a DELETE kérés	Törlés és a DELETE kérés	
83.	Törlés megszakítása	Törlés megszakítása	
84.	Törli a kérést	Törli a kérést	
85.	A jelszó visszaállítás folyamata	A kérést reseteli és újratölti az oldalt	
86.	A jelszó visszaállítása sikertelen kezelése	A jelszó visszaállítása sikertelen kezelése	
87.	visszaGomb van legalul	visszaGomb van legalul	
88.	meghívja az oldalTakarito-t és lekéri a szükséges adatokat	meghívja az oldalTakarito-t és lekéri a szükséges adatokat	

89.	vendég felhasználó esetén angol nyelven is működik	vendég felhasználó esetén angol nyelven is működik	
90.	bejelentkezett felhasználó esetén API-ból kéri le az adatokat	bejelentkezett felhasználó esetén API-ból kéri le az adatokat	
91.	meghívja a dekor_vonal_blokk al-t kétszer	meghívja a dekor_vonal_blokk al-t kétszer	
92.	visszaGomb hozzáadása a megfelelő szöveggel	visszaGomb hozzáadása a megfelelő szöveggel	
93.	hiba esetén, ha a vendég nyelv ismeretlen, kivételt dob	hiba esetén, ha a vendég nyelv ismeretlen, kivételt dob	

A unit teszteket a játékon kívül lefuttattuk átlagosan 2-3 másodpercet igényelt. A Unit táblázat tartalma rövidebb, mert az API-knál külön tesztnek érzékel néhány hiba kezelést a táblázatban.



```
Test Suites: 10 passed, 10 total
Tests:      115 passed, 115 total
Snapshots:  0 total
Time:       2.201 s
```

11. ábra: Unit tesztek

A játékot és a weboldalt ezeken kívül a Sonarqube segítségével is optimalizáltuk, ellenőriztük, ahol volt időnk és lehetőségünk. Viszont, ahogy a Githubban található issuekban látható a kognitív komplexitás szabályait nem mindenhol tudtuk betartani.

A játékokat többféle szempontból is kell tesztelni. Megtettünk mindent részünkről amit lehetett. A funkcionalitást, a kombinatorikát, a játék haladást, kompatibilitást, a regressziót, a UI-t, a hatékonyságot és az integrációt is mind tesztenünk kellett.

Funkcionalitás szempontjából többször mindkét lehetséges ending-ig végigvittük, hogy láthassuk, hogy lehetséges-e egyáltalán. Ezzel együtt tudtuk tesztelni, hogy a mozgás kombinatorikája, az animációk kezelése, a játék haladási ciklusát és a UI-t teszteltük.

Programozás közben állandóan teszteltük az új funkciókat, hogy a régiekkel működnek, avagy regresszív tesztelést hajtottunk végre.

Ezekon kívül külsős felhasználókkal is teszteltünk.

Továbbfejlesztési lehetőségek

A dokumentáció során eddig a programunk jelenlegi állapotáról olvashattak, de most már kitérnénk lehetséges fejlesztésekre amikre nem volt lehetőségünk, más terveket és vagy ötleteket kifejtenénk.

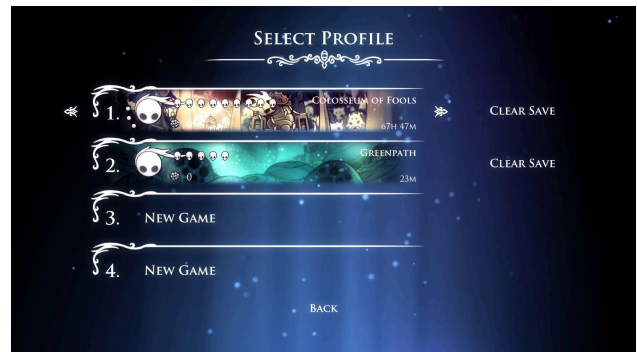
Elsősorban kisebb kiegészítésekkel kezdeném:

Egy rendes parallax háttereket csinálni (lásd: 12. ábra). Ennek megoldás nagyobb grafikai teljesítményt kérne és sokkal több időt és tervezést. Mi csak a layereket (rétegeket) hozunk létre, hogy időben lehessen teljesíteni a záródolgozatot.



12. ábra: Példa a parallax háttére

A mentés kezelésnél a létező mentések megkapják a mentés szobájának a hátterét és adatait. Apróság, de vizuálisan sokat dob a programon. A mentés szobáját és a mentésen található játékos adatokat kell figyelni (lásd: 13. ábra). A játék resetelésének lehetősége (lásd: 13. ábra).



13. ábra: Példa a mentés kezelésre

Vizualitás szempontjából az egész programot lehet esztétikusabbá tenni és kiegészíteni más nyelvekkel.

A játékon belül nagyobb szobákkal, több ellenséggel, rejtvények és több titkok hozzáadásával érdekesebbé tenni a világot. Példaként a Hollow Knight térképe (lásd: 14. ábra), körülbelül 3 évnyi munka következménye +160 különféle ellenséggel és még többnyi NPC-vel és érdekességekkel



14. ábra: Hollow Knight teljes pálya

A játékmenet diverzifikálása érdekében több képességet is tennék a játékban, akár a Hollow Knight széria nyomában, akár új dolgokkal előállni. Új animációkat igényelnek ezek a részek és nem tudom, hogy a kaboom limitjein belül lehetséges-e egyáltalán. Abban az esetben egy kompatibilis könyvtárral kell dolgozni, vagy egy teljesen másikat használni, ha Kaboom limitál.



15. ábra: Hollow Knight képességek

A kognitív komplexitás csökkentése, optimalizálás és a felesleges fonalak rendes kezelése és lezárása. Jelenleg a játékunkban előfordulnak olyan elemek, amik a sok aszinkron működés és fonalra osztása túlterhelte a rendszert. Szerencsére ez nem gyakori hiba, a játék ettől függetlenül játszható marad. Az offscreen (képernyőn kívüli) nem használt dolgok kikapcsolása, teljes nem használata.

Jelenlegi létező bugok kezelése, mint például, hogy bizonyos esetekben át lehet esni objektumokon, mert nem tölt be rendesen az objektum vagy a collision nem jól érzékelődik.

Pixel pontosítások, hogy ne lebegjen a macska néhány mappon.

A játék telefonos verzió 100%-os elkészítése.

A képregény feldolgozása teljesen. A jelenlegi játék csak az első képregényt és egy apró részletet a 44. képregény részéből. A sorozat 57 résszel és 2 darab kiegészítéssel rendelkezik. A sorozat több külön műfajt is feldolgoz, ami külön-külön játékokat is akár megérdemelve, köztük: sci-fi kaland, úrkutatás, karakterközpontú dráma és vígjáték, pszichológiai horror, thriller, társadalmi és filozófia felfedezés témakörökkel foglalkoznak főként (megér egy olvasást 📖(🌀🌀)).

Felhasználói dokumentáció

A program célja és lényegesebb funkciói

A program célja, hogy szórakoztassuk az embereket (módjuk ilyen játék esetén ez emberfüggő) és hogy demonstráljuk az új dolgokat amiket tanultunk az évek alatt.

A programnak a lényegesebb funkciói (ezek részletezve lesznek a program részletes bemutatása pontban):

- Sütik
- Felhasználó kezelés
- Játék

Szükséges hardverek és szoftverek

Ismert minimum hardware igény (ennél gyengébb gépekkel nem teszteltünk, nem tudjuk garantálni a program hatékony futását, ha egyáltalán le tud futni):

64 bites processzor és operációs rendszer

Op. rendszer: Windows 10 version 21H1 (build 19043) vagy újabb

Processzor: Intel(R) Core(TM) i5-14500 (2.60 GHz)

Memória: 16 GB RAM

Grafika: integrált GPU

DirectX: Verzió: 12

Tárhely: 900 MB szabad hely

Igényelt perifériák:

- Billentyűzet
- Monitor/kivetítő
- Egér/érintőfelület

Igényeltelt szoftverek:

- Visual Studio Code
- Node.js
- XAMPP

Hibaelhárítás és gyakori kérdések

A program nem indul el

- XAMPP nem fut – Ellenőrizze, hogy az Apache és a MySQL szolgáltatás zöld-e a XAMPP vezérlőpulton.
- Node.js nincs telepítve – Gépelje be parancssorba, hogy `node -v`. Ha hibát ír, telepítse a Node.js-t a fent leírtak szerint.
- Tűzfal vagy vírusirtó blokkolja – Adjon kivételt a `node.exe` és a böngésző számára.
- Hibás port – Ha a :3000-es port foglalt, állítson be másikat a konfigurációs fájlban (kérdés esetén lásd a támogatást).

Grafikai akadozás, alacsony FPS

- Frissítse a videokártya-drivert a gyártó oldaláról.
- Ellenőrizze, hogy a DirectX 12 telepítve van-e (írja be a Start menübe: `dxdiag`).

Bejelentkezési problémák

- Győződjön meg róla, hogy a MySQL fut (XAMPP).
- Ellenőrizze az e-mail címet és a jelszót; próbálja ki a jelszó-helyreállítást.

Ha továbbra sem működik, forduljon a fejlesztőkhöz.

Telepítés és indítás lépései

A fentiek alapján a gépünk le tudja futtatni a programunkat, akkor a következőképpen kell telepíteni és elindítani a játékot:

16. Telepítsük a Visual Studio Code-ot vagy más hasonló program futtatásra alkalmas applikációt.

17. Telepítsünk egy lokális adatbázis rendszert kezelő applikációt (pl.: XAMPP, Oracle developer, stb.).

18. Töltsük le a Node.js-t a <https://nodejs.org/en> oldalról.

(A gépeken feltételezhetően ez az első három lépés már meg van csinálva, de ellenőrizni kell)

19. Töltsük le a Githubról a zárodolgozatot a következő linkről:
<https://github.com/nagyhuszarvirag/Vizsgaremek-Metroidvania-game.git>

20. A tömörített mappát bontsuk ki és helyezzük egy számunkra könnyen megtalálható mappába.

21. Az első lépésben letöltött (,mostantól Visual Studio Code) applikációban megnyitjuk a kibontott mappát.

22. A Visual Studio Code-ban nyissunk egy új terminált.

23. A terminálba írjuk bele, hogy `npm init` abban az esetben, ha nincs `package.json` fájl

24. A terminálba írjuk bele, hogy `cd backend`

a. Játék esetén a következőket kell még a terminálba beírni (, ha először futtatni a programot és nincsen minden telepítve):

i. `npm install`

ii. `npm install bcrypt`

b. Az `npm run start`-ot beírjuk a terminálba a program futtatásáért

25. Miután ez megtörtént a XAMPP vagy az általunk választott adatbázist elindítjuk lokálisan és a zárodolgozat `backend/sql` mappájában található adatbázis SQL-t betöltjük.

26. A játékot ezek után lehet futtatni a következőkkel (ezt is a Visual Studio Code terminálba kell beírni): `npm run start`

27. A következő linken lesz található az oldal ettől a ponttól: <http://127.0.0.1:3000>

28. Az oldal használható vendédként és fiókkal is, viszont vendédként kevesebb dolog elérhető, és teljes kilépés után minden mentés elvész.
29. Használati lépésekért a következő fő pontot olvassa el: “A program részletes bemutatása”

A program részletes bemutatása

A sikeres telepítés és indítás után elkezdhetjük használni a programot. Alapvetően egy vendég felhasználóként indulunk, de bejelentkezés és vagy regisztráció után több funkcióhoz férünk hozzá. A következőkben az oldal összes felhasználói funkcióján végigmegyünk és grafikusán bemutatjuk, hogy mi hogyan és miért van. A bemutatás olvasása közben az az alap feltételezés, hogy mindkét felhasználó hozzáfér a funkcióhoz, kivéve, ha külön kijelentjük, hogy ez a vendégnél nem elérhető vagy csak másképpen elérhető.

A fő oldalunkon egyből több lehetséges funkció van. Balról-jobbra és fentről-lefelé a következők:

- Sütik
 - Ezzel hozzuk létre a vendéget és ezzel kezelünk minden felhasználói dolgot lokálisan, kritikus az oldal működéséhez
- Zene ki- és be kapcsolása (a játékon kívül csak egy szám érhető el)
- Regisztráció/bejelentkezés, ha vendégek vagyunk és kijelentkezés, ha bejelentkezett felhasználók vagyunk
 - Regisztráció/bejelentkezés esetén még lehet igényelni egy jelszó resetet, ha elfelejtettük a jelszavunkat
- Játékok indítása
 - A menüpont a 4 mentéshez visz, ami elérhető a játékban
- Beállítások
 - 2 lehetséges választás van itt:
 - Általános beállítások
 - Hangerő állítása
 - Nyelv beállítása
 - Telefonos mód beállítása

- Mozgás és interakciók gombjainak beállítása
- Beállítások mentése (lokálisan a vendégnél és az adatbázisban a felhasználónál)
- Fiókkezelés (csak bejelentkezett felhasználó éri el)
 - Fiók adat (e-mail és felhasználónév) változtatás és fiók törlése lehetséges itt egy sima felhasználónak
 - Az adminoknak itt található az adminok panelje is
 - Jelenlegi felhasználók fiók adatainak változtatása (e-mail és felhasználónév és jogszint)
 - Elfelejtett jelszó kérelmek és állapotuk
- Kreditek
 - A készítőik és fontosabb közreműködők bemutatása
- Trófeák
 - A fiókhoz/vendéghez kötött teljesítmények, ha nem értük el őket, akkor fel kell fedni őket, hogy lehessen látni a tartalmát (az akaratunk elleni spoilerek elkerülése érdekében)
- Kilépés
 - Törli a sessiont és a localhostból az adatainkat (és bizonyos esetekben be is zárja az ablakot)

A játék és annak menete

A Metroidvaniákban az a jellemző, hogy ahogy haladunk, úgy tudjuk meg, hogy hova megyünk és mi a cél. Gyakori, hogy vissza kell menni egy régebbi helyre, amikor új képességünk van, hogy a pálya egy eddig felfedezetlen részét elérhessük. A felfedezés, a környezet, a történet és a cél megismerése a stílus lényege. A klasszikus platformerektől és más játék stílusoktól még az különbözteti meg, hogy a játék egy nem lineáris rendszerrel rendelkezik. Nincsen jelen az, hogy ha az első szintet kivittük, akkor oda nem mehetünk vissza. Nincsen első szint (a klasszikus értelemben)! Játék szakaszok vannak, ami a pálya változásához vezethet, (pl.: a fertőzés elterjedése, ahogy haladunk a játékkal előre és egy ponton egy teljes pályaszakasz elérhetetlen a fertőzés miatt,) ami ebben a játékban nincsen jelen.

A játék egy jelenettel (mostantól cutscene), kezdődik, ami bemutatja a játékost az alap kontextusba. A játékos ezek után a kezdő szobában köt ki, ahol teljes biztonságban kipróbálhatja az alap mozgási- támadási- és interakciós mechanikákat. A szobában található még egy NPC (magyarázat: Non-player character, a játékokban egy gyakori eszköz, hogy történetet meséljenek, lakottá tegyenek helyeket a játékban és vagy más fejlesztők által kitalál játékos interakciókra van használva), akivel lehet beszélni. A szobából egy kijárat található, ami a Mitteous fennsíkra visz ki minket. Itt több átjáró is található más-más szobákba. A játékosnak a feladata, hogy felfedezze őket. A pályákon találhatóak a következők: rejtett szobák, széttörhető falak, szakadékok, beszakadó talaj, főellenségek (boss fights), kisebb ellenségek, NPC interakciók, pálya interakciók és különleges képességek.

Jó játékot! ☐^ . ~.^☐

Csapatmunka megvalósítása

A csapatunkban 2 csapatagból áll. Mindketten a fejlesztői, tervezői és tesztelői szerepet töltöttük be, ennek a felosztása a következőképpen történt meg (és más lent részletezett munkáink):

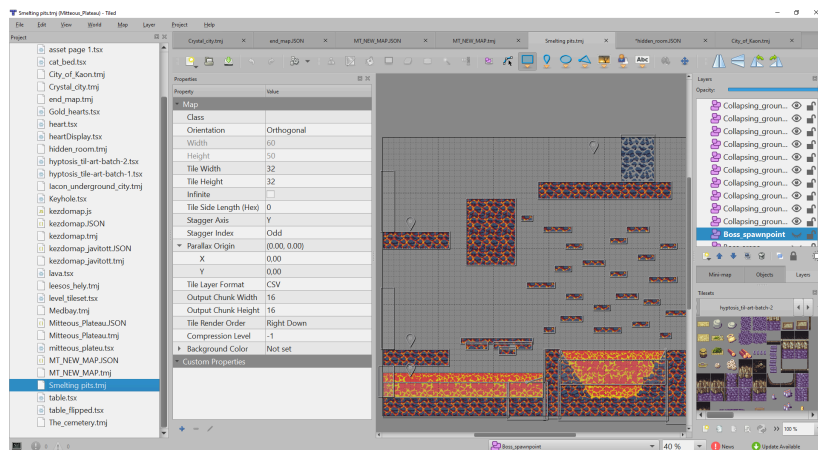
Nagy-Huszár Virág:

- **JSON nyelvkezelés**

A játék 2 nyelven (magyarul és angolul) elérhető és emiatt a szövegezést JSON-ben tároljuk, hogy minden új nyelvet amit hozzáadnánk, az könnyen kezelhető legyen

- **JSON szobák és grafikusan a szobák létrehozása**

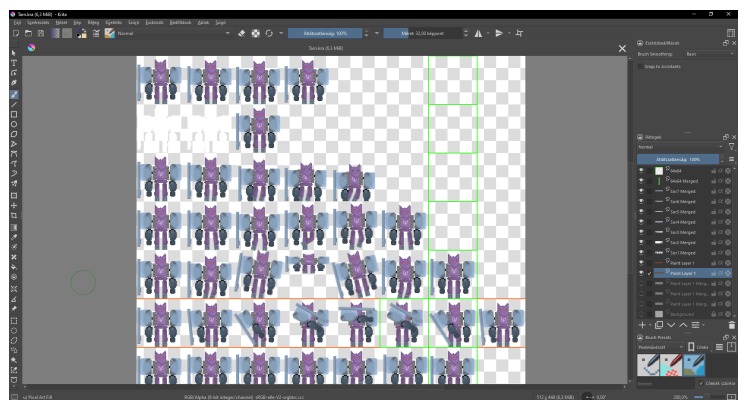
A Kritában való tile megrajzolástól, a Tiled-on belüli teljes megrajzolás és játékon belüli definiálásig



16.ábra: Tiled applikáció

- **Sprite-ok létrehozása és kezelése (NPC-k, a játékos és ellenségek)**

A Kritában való sima sprite megrajzolástól, a teljes spritesheet megrajzolásán keresztül a játékon belüli definiálásig



17.ábra: Krita applikáció

- **NPC interakciók és viselkedés kezelése**

A jelenetek, az endingek az interakciók alapján

- **Start menü, trófeák menü, kreditek menü elkészítése**

Mentésből meghívás a guestnek és a fiókos usernek

- **Adatbázis kiegészítése, optimalizálás és fejlesztése**

Az adatbázis szintakszis hibáit, logikai hibáit kezeltem és optimalizáltam az adatok tárolását (pl.: a nyelveknek külön táblát adtam). Fejlesztés alatt az új táblák, triggerek és kapcsolatokat hoztam létre.

- **Zene keresés és kezelés**

A játékokból és filmekből és sorozatokból a hanganyag kivétele

- **Videóanyag megvágása és kódban kezelése**

- **Mozgás kezelése**

Dinamikus mozgáskezelés a beállítások alapján

- **Minden más grafikai igényű dolog keresés, kialakítása és kódban való kezelése**

Információk összeszedése az itch.io-ról, a Pinterestről és a játékokból (Hollow Knight, Hollow Knight: Silksong és Transformers: Fall of Cybertron)

- **Optimalizálás**

A programban redundáns kód minimalizálása és kognitív komplexitás csökkentése

- **Tesztelés (manuális és unit)**

A program unit tesztelése a Jest nevezetű környezettel és a játék manuális tesztelése. A külsős játékos tesztelés megszervezése és végrehajtása

- **Bejelentkezés és fiókkezelés**

A játék lehetőséget ad regisztrációra és bejelentkezésre, ahol a felhasználók adatai biztonságosan kerülnek eltárolásra, és a játék mindig a megfelelő felhasználó adataival indul el.

- **Süтик és minden azzal kapcsolatos dolog**

A játék indulásakor egy süti elfogadó ablak jelenik meg, amely megjegyzi a felhasználó választását, és ha nincs bejelentkezett felhasználó, egy vendég profilt hoz létre.

- **Beállítások menü, kilépés menü és be- és kilépés menü kezelése**

A különböző menük (beállítások, játék közbeni menü, kilépés megerősítése) programból készülnek el, és lehetőséget adnak a játék irányítására és testreszabására.

- **Mentés kezelés**

A játék menti a játékos előrehaladását, például a pozícióját, megszerzett képességeit és döntéseit, így később ugyanonnan lehet folytatni a játékot.

- **Szobák összekötése és töltése**

A játék több különálló pálya részből áll, amelyek között átjárókon keresztül lehet közlekedni, és minden új terület külön töltődik be.

- **Kamera kezelés**

A kamera követi a játékost mozgás közben, figyelembe veszi az irányt és az ugrásokat, valamint nem engedi, hogy a pálya határain kívülre lássunk.

- **Játékos logika és támadások**

A játékos mozgása és képességei (például ugrás, támadás, speciális mozdulatok) külön szabályok alapján működnek, és ezek együtt adják a játékmenet alapját.

- **Ellenségek, scrapletek, Tarn és Sparkeater**

A játékban található egyszerűbb ellenfelek és erősebb bossok, amelyek különböző módon támadnak, és eltérő kihívást jelentenek a játékos számára.

- **HP kezelés**

Az életerő rendszer figyeli a játékos sebzését és gyógyulását, valamint kezeli a halált és az újraindulást.

- **Játék UI-ja**

A játék felülete megjeleníti például az életerőt, képességek állapotát és különböző információkat, amelyek segítik a játékost.

- **Hibajavítás**

A fejlesztés során folyamatosan javítottam a hibákat és finomítottam a rendszereket, hogy a játék stabilan és megfelelően működjön.

Összefoglalás

Őrült ötletek ide, őrült ötletek oda. Elkezdtünk valamit, amiről semmit sem tudtunk, és most már egy felvállalható terméket adunk ki kezeinkből. Sok különböző sapkát viseltünk és megérte. Kelleténél többször néztünk, hogy hogyan és miért nem működik valami, de egymást kisegítettük szükség esetén, mert a mesterséges intelligencia jelenleg nem képes rendesen segíteni a játék fejlesztésében. A bevezetésünkben kitűzött célunkat teljesítettük 100%-ban mind szakmai és mind személyes szempontból.

Abban az esetben, ha a játékban talált jelenetek felkeltették az érdeklődését a képregénnyel kapcsolatban személyesen tudom ajánlani a The Transformers: More than Meets the Eye (ezt vettük alapul a játékhoz) képregény sorozatot és minden más IDW képregény által kiadott Transformers képregényt is.

Ábrajegyzék

- 1. ábra: SQL adatszerkezet
- 2. ábra: Nyelvkezelés JSON-ök
- 3. ábra: Iacon városa
- 4. ábra: Scraplet
- 5. ábra: Sparkeater boss
- 6. ábra: Kaon városa
- 7. ábra: Crystal city
- 8. ábra: Az olvasztó gödrök
- 9. ábra: Tarn
- 10. ábra: Lost Light hajó
- 11. ábra: Unit tesztek
- 12. ábra: Példa a parallax háttére
- 13. ábra: Példa a mentés kezelésre
- 14. ábra: Hollow Knight teljes pálya
- 15. ábra: Hollow Knight képességek
- 16. ábra: Tiled applikáció
- 17. ábra: Krita applikáció

Irodalomjegyzék

<https://stackoverflow.com/questions/8057802/html-button-close-window> (utolsó megtekintés: 2026.02.22.)

<https://www.mapeditor.org/> (utolsó megtekintés: 2026.02.22.)

<https://hu.wikipedia.org/wiki/Metroidvania> (utolsó megtekintés: 2026.03.10.)

<https://en.wikipedia.org/wiki/Metroidvania> (utolsó megtekintés: 2026.03.10.)

<https://kaboomjs.com/> (utolsó megtekintés: 2026.03.21.)

<https://itch.io/c/7222502/vizsgaremek> (utolsó megtekintés: 2026.04.12.)

<https://www.youtube.com/watch?v=UWHiMgFjRrU> (utolsó megtekintés: 2026.04.16.)

<https://codepen.io/shubniggurath/pen/oPGyQw> (utolsó megtekintés: 2026.04.16.)

<https://codepen.io/braedencraig/pen/dBVKzO> (utolsó megtekintés: 2026.04.16.)

<https://www.youtube.com/watch?v=H0SE4x5ZfYc> (utolsó megtekintés: 2026.04.16.)

<https://www.youtube.com/@InboundShovel/shorts> (itt körülbelül minden kellett, mert nagyon jó tanácsokat kaptunk) (utolsó megtekintés: 2026.04.16.)

<https://chat.deepseek.com/share/ptqwsh0c204fbka5yc> (utolsó megtekintés: 2026.04.17.)

<https://github.com/mifi/lossless-cut> (utolsó megtekintés: 2026.04.18.)

<https://tudasbazis.sulinet.hu/hu/szakkepzes/informatika/programozaselvelet/a-programozas-folyamata-kodolas-tesztes-es-dokumentacio-feladata-fobb-jellemzoi/felhasznaloi-dokumentacio> (utolsó megtekintés: 2026.04.22.)

https://store.steampowered.com/app/367520/Hollow_Knight/ (utolsó megtekintés: 2026.04.22.)

https://progalap.elte.hu/downloads/seged/eTananyag/lecke28_lap1.html (utolsó megtekintés: 2026.05.03.)

<https://jestjs.io/docs/getting-started> (utolsó megtekintés: 2026.05.03.)

<https://krita.org/en/> (utolsó megtekintés: 2026.05.03.)

<https://www.youtube.com/watch?v=UWHiMgFjRrU> (utolsó megtekintés: 2026.05.03.)

<https://stackoverflow.com/questions/8057802/html-button-close-window> (utolsó megtekintés: 2026.05.03.)

https://www.reddit.com/r/GameDevelopment/comments/1jmijh7/what_is_the_best_way_to_go_about_coding_multiple/ (utolsó megtekintés: 2026.05.03.)

<https://tfwiki.net/wiki/Scraplet> (utolsó megtekintés dátuma: 2026.05.04.)

https://tfwiki.net/wiki/Lost_Light (utolsó megtekintés dátuma: 2026.05.04.)

https://tfwiki.net/wiki/The_Transformers:_More_than_Meets_the_Eye (utolsó megtekintés dátuma: 2026.05.04.)

https://tfwiki.net/wiki/The_Not_Knowing (utolsó megtekintés dátuma: 2026.05.04.)

https://tfwiki.net/wiki/Liars,_A_to_D_Part_1:_How_to_Say_Goodbye_and_Mean_It (utolsó megtekintés dátuma: 2026.05.04.)